

MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers

Lili Yu^{*1} Dániel Simig^{*1} Colin Flaherty^{*2} Armen Aghajanyan¹ Luke Zettlemoyer¹ Mike Lewis¹

Abstract

Autoregressive transformers are spectacular models for short sequences but scale poorly to long sequences such as high-resolution images, podcasts, code, or books. We propose MEGABYTE, a multi-scale decoder architecture that enables end-to-end differentiable modeling of sequences of over one million bytes. MEGABYTE segments sequences into patches and uses a *local* submodel within patches and a *global* model between patches. This enables sub-quadratic self-attention, much larger feedforward layers for the same compute, and improved parallelism during decoding—unlocking better performance at reduced cost for both training and generation. Extensive experiments show that MEGABYTE allows byte-level models to perform competitively with subword models on long context language modeling, achieve state-of-the-art density estimation on ImageNet, and model audio from raw files. Together, these results establish the viability of tokenization-free autoregressive sequence modeling at scale.

1. Introduction

Sequences of millions of bytes are ubiquitous; for example, music, image, or video files typically consist of multiple megabytes. However, large transformer decoders (LLMs) typically only use several thousand tokens of context (Brown et al., 2020; Zhang et al., 2022a)—both because of the quadratic cost of self-attention but also, more importantly, the cost of large feedforward networks per-position. This severely limits the set of tasks where LLMs can be applied.

We introduce MEGABYTE, a new approach to modeling long byte sequences. First, byte sequences are segmented into fixed-sized patches, loosely analogous to tokens. Our model then consists of three parts: (1) a *patch embedder*,

^{*}Equal contribution

¹Meta AI.

²Augment Computing. Work performed while at Meta AI. Correspondence to: Lili Yu <liliyu@meta.com>, Mike Lewis <mikelewis@meta.com>.

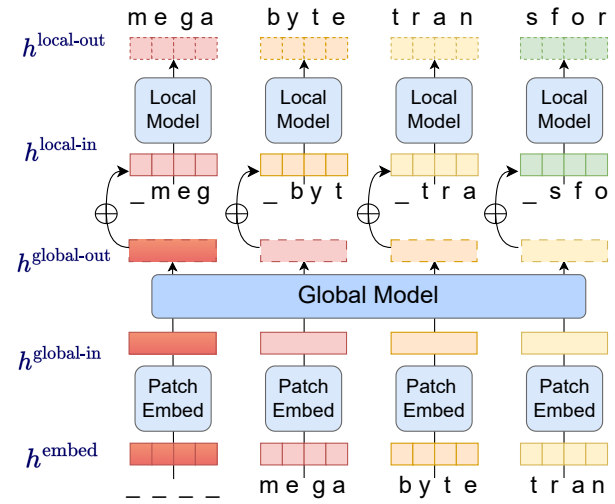


Figure 1. Overview of MEGABYTE with patch size $P = 4$. A small *local* model autoregressively predicts each patch byte-by-byte, using the output of a larger *global* model to condition on previous patches. Global and Local inputs are padded by P and 1 token respectively to avoid leaking information about future tokens.

which simply encodes a patch by losslessly concatenating embeddings of each byte, (2) a *global* module, a large autoregressive transformer that inputs and outputs patch representations and (3) a *local* module, a small autoregressive model that predicts bytes within a patch. Crucially, we observe that for many tasks, most byte predictions are relatively easy (for example, completing a word given the first few characters), meaning that large networks per-byte are unnecessary, and a much smaller model can be used for intra-patch modelling.

The MEGABYTE architecture gives three major improvements over Transformers for long sequence modelling:

- Sub-quadratic self-attention** Most work on long sequence models has focused on mitigating the quadratic cost of self-attention. MEGABYTE decomposes long sequences into two shorter sequences, and optimal patch sizes reduces the self-attention cost to $O(N^{\frac{4}{3}})$, which remains tractable for even long sequences.

- Per-patch feedforward layers** In GPT3-size mod-

MEGABYTE模型：使用多尺度变换器处理百万字节级序列数数量

Lili Yu^{*1} Dániel Simig^{*1} Colin Flaherty^{*2} Armen Aghajanyan¹ Luke Zettlemoyer¹ Mike Lewis¹

摘要

Autoregressive transformers are spectacular models for short sequences but scale poorly to long sequences such as high-resolution images, podcasts, code, or books. We propose MEGABYTE, a multi-scale decoder architecture that enables end-to-end differentiable modeling of sequences of over one million bytes. MEGABYTE segments sequences into patches and uses a *local* submodel within patches and a *global* model between patches. This enables sub-quadratic self-attention, much larger feedforward layers for the same compute, and improved parallelism during decoding—unlocking better performance at reduced cost for both training and generation. Extensive experiments show that MEGABYTE allows byte-level models to perform competitively with subword models on long context language modeling, achieve state-of-the-art density estimation on ImageNet, and model audio from raw files. Together, these results establish the viability of tokenization-free autoregressive sequence modeling at scale.

1. 引言

包含数百万字节序列的数据极为常见；例如，音乐、图片或视频文件通常都有数兆字节的大小。然而，大型Transformer解码器（即大语言模型）通常仅能处理数千个上下文标记（Brown等人，2020年；Zhang等人，2022a）——这既是由于自注意力机制的二次方计算成本，更重要的是由于大型前馈网络在每个位置上的高计算开销。这些因素严重限制了大语言模型可被应用的任务范围。

我们推出了MEGABYTE，这是一种用于建模长字节序列的新方法。首先，这些字节序列会被分割成固定大小的补丁，其结构与标记大致相似。该模型随后由三部分组成：(1)一个补丁嵌入器，

^{*}同等贡献者：¹Meta AI的²增强计算团队。相关研究工作是在Meta AI期间完成的。联系方式：LiliYu <liliyu@meta.com>，Mike Lewis <mikelewis@meta.com>。

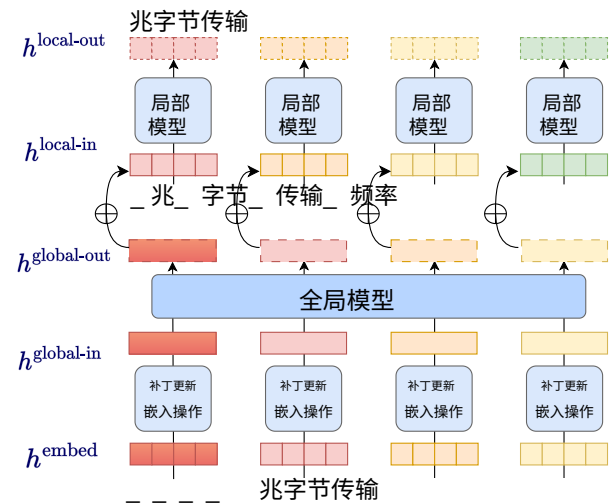


图1。采用 $P = 4$ 块大小的MEGABYTE模型概览。一个较小的局部模型通过自回归方式逐字节预测每个数据块，同时利用较大的全局模型的输出作为条件来参考之前的数据块。为防止泄露未来标记的信息，全局输入和局部输入分别会用 P 个填充字符和1个标记进行补齐。

其实现方式简单，即为每个字节对应的嵌入操作进行无损拼接以编码补丁；(2) 一个全局模块，即一个大型自回归Transformer架构，负责输入和输出补丁的表示形式；以及(3)一个局部模块，即一个小型自回归模型，用于预测补丁内的各个字节。至关重要的是，我们发现对于许多任务而言，大多数字节预测都相对容易（例如，根据前几个字符推断出完整单词），这意味着无需为每个字节都使用大型网络，而是可以用体积小得多的模型来完成补丁内部的建模工作。

MEGABYTE模型在长序列建模方面相较于Transformer架构带来了三项重大改进：

- 次二次方自注意力机制** 大多数关于长序列模型的研究都致力于降低自注意力机制的二次方级计算成本。MEGABYTE模型将长序列拆分为两个较短的序列，再通过最优的块大小将自注意力的计算成本降至 $O(N^{\frac{4}{3}})$ ，即便面对极长的序列也能保持可行的计算效率。

- 每个补丁对应的前馈层** 在GPT3规模级别的模型中—

els, more than 98% of FLOPS are used in computing position-wise feedforward layers. MEGABYTE uses large feedforward layers per-patch rather than per-position, enabling much larger and more expressive models for the same cost. With patch size P , where a baseline transformer would use the same feedforward layer with m parameters P times, MEGABYTE can use a layer with mP parameters once for the same cost.

- Parallelism in Decoding** Transformers must perform all computations serially during generation because the input to each timestep is the output from the previous timestep. By generating representations for patches in parallel, MEGABYTE allows greater parallelism during generation. For example, a MEGABYTE model with 1.5B parameters can generate sequences 40% *faster* than a standard 350M Transformer, whilst also improving perplexity when trained with the same compute.

Together, these improvements allow us to train much larger and better-performing models for the same compute budget, scale to very long sequences, and improve generation speed during deployment.

MEGABYTE also provides a strong contrast to existing autoregressive models that typically use some form of tokenization, where sequences of bytes are mapped to larger discrete tokens (Sennrich et al., 2015; Ramesh et al., 2021; Hsu et al., 2021). Tokenization complicates pre-processing, multi-modal modelling, and transfer to new domains, while hiding useful structure from the model. It also means that most state-of-the-art models are not truly end to end. The most widely used approaches to tokenization require language-specific heuristics (Radford et al., 2019) or lose information (Ramesh et al., 2021). Replacing tokenization with efficient and performant byte models would therefore have many advantages.

We conduct extensive experiments for both MEGABYTE and strong baselines. We use a fixed compute and data budget across all models to focus our comparisons solely on the model architecture rather than training resources, which are known to benefit all models. We find that MEGABYTE allows byte-level models to perform competitively with sub-word models on long context language modeling, achieve state-of-the-art perplexities for density estimation on ImageNet, and allow audio modelling from raw audio files. Together, these results establish the viability of tokenization-free autoregressive sequence modeling at scale.

2. MEGABYTE Transformer

2.1. Overview

MEGABYTE is an autoregressive model for efficiently modeling long input sequences. MEGABYTE is comprised of

3 components: (1) a *patch embedder* that inputs a discrete sequence, embeds each element, and chunks it into patches of length P (2) a large *global* Transformer that contextualizes patch representations by performing self-attention over previous patches, and (3) a smaller *local* Transformer that inputs a contextualized patch representation from the global model, and autoregressively predict the *next* patch.

2.2. Components

Patch Embedder with patch size of P maps a byte sequence $x_{0..T}$ to a sequence of patch embeddings of length $K = \frac{T}{P}$ and dimension $P \cdot D_G$.

First, each byte is embedded with a lookup table $E^{\text{global-embed}} \in \mathbb{R}^{V \times D_G}$ to an embedding of size D_G and positional embeddings are added.

$$h_t^{\text{embed}} = E_{x_t}^{\text{global-embed}} + E_t^{\text{pos}} \quad t \in [0..T] \quad (1)$$

Then, byte embeddings are reshaped into a sequence of K patch embeddings with dimension $P \cdot D_G$. To allow autoregressive modelling, the patch sequence is padded to start with a trainable patch-sized padding embedding ($E^{\text{global-pad}} \in \mathbb{R}^{P \times D_G}$), and the last patch is removed from the input. This sequence is the input to the global model, and is denoted $h^{\text{global-in}} \in \mathbb{R}^{K \times (P \cdot D_G)}$.

$$h_k^{\text{global-in}} = \begin{cases} E^{\text{global-pad}}, & \text{if } k = 0, \\ h_{((k-1) \cdot P):(k \cdot P)}^{\text{embed}}, & k \in [1, \dots, K), \end{cases} \quad (2)$$

Global Model is a decoder-only Transformer with dimension $P \cdot D_G$ that operates on a sequence of K patches. It incorporates a self-attention mechanism and causal masking to capture dependencies between patches. It inputs a sequence of K patch representations $h_{0:K}^{\text{global-in}}$, and outputs an updated representation $h_{0:K}^{\text{global-out}}$ by performing self-attention over previous patches.

$$h_{0:K}^{\text{global-out}} = \text{transformer}^{\text{global}}(h_{0:K}^{\text{global-in}}) \quad (3)$$

The output of the final global layer $h_{0:K}^{\text{global}}$ contains K patch representations of dimension $P \cdot D_G$. For each of these, we reshape them into sequences of length P and dimension D_G , where position p uses dimensions $p \cdot D_G$ to $(p+1) \cdot D_G$. Each position is then projected to the dimension of the local model with a matrix $w^{\text{GL}} \in \mathbb{R}^{D_G \times D_L}$ where D_L is the local model dimension. We then combine these with byte embeddings of size D_L for the tokens in the *next* patch $E_{x_{(k \cdot P + p - 1)}}^{\text{local-embed}}$. The local byte embeddings is offset by one with a trainable local padding embedding ($E^{\text{local-pad}} \in \mathbb{R}^{D_L}$)

在这类模型中，超过98%的浮点运算次数都用于计算各位位置的前馈层。MEGABYTE模型采用按补丁划分的大型前馈层，而非按位置划分，因此在相同的成本下就能构建出规模更大、表达能力更强的模型。当块大小为 P 时，若基于常规Transformer架构需要使用参数量为 m 的前馈层且需使用 P 次，那么MEGABYTE模型则只需使用一次参数量为 mP 的前馈层即可达到同样的效果。

- 解码过程中的并行性**由于每个时间步的输入都是前一个时间步的输出，Transformer在生成内容时必须以串行方式执行所有计算。而MEGABYTE模型能够通过并行处理各个补丁的表示，从而在生成过程中实现更高的并行度。例如，一个拥有15亿参数的MEGABYTE模型生成序列的速度比标准的3.5亿参数模型快40%，并且在使用相同的计算资源进行训练时还能降低困惑度。

通过这些改进的协同作用，我们能够在相同的计算预算下训练出规模更大、性能更优的模型，支持处理极长的序列，并在部署时提升生成速度。

MEGABYTE also provides a strong contrast to existing autoregressive models that typically use some form of tokenization, where sequences of bytes are mapped to larger discrete tokens (Sennrich et al., 2015; Ramesh et al., 2021; Hsu et al., 2021). Tokenization complicates pre-processing, multi-modal modelling, and transfer to new domains, while hiding useful structure from the model. It also means that most state-of-the-art models are not truly end to end. The most widely used approaches to tokenization require language-specific heuristics (Radford et al., 2019) or lose information (Ramesh et al., 2021). Replacing tokenization with efficient and performant byte models would therefore have many advantages.

我们对MEGABYTE模型以及多种强基线模型都进行了大量实验。我们在所有模型中均采用固定的计算与数据预算，这样就能将比较重点完全放在模型架构上，而非训练资源——因为众所周知，训练资源会对所有模型都有益处。实验结果表明，MEGABYTE模型能够让字节级模型在长上下文语言建模任务中展现出与次词级模型相当的性能，在ImageNet的密度估计任务中实现最先进的困惑度指标，并且还能直接对原始音频文件进行音频建模。综上所述，这些成果充分证明了大规模应用无需标记化的自回归序列建模的可行性。

2. MEGABYTE模型Transformer架构

2.1. 概述

MEGABYTE模型是一种自回归模型，可用于高效地对长输入序列进行建模。该模型由多个组件构成。

具体而言，它包含3个组成部分：(1)一个补丁嵌入器，用于输入离散序列、对每个元素执行嵌入操作，并将其分割成长度为 P 的补丁；(2)一个规模较大的全局Transformer架构，通过针对先前生成的补丁应用自注意力机制来为这些补丁表示赋予上下文信息；以及(3)一个规模较小的局部Transformer架构，它接收来自全局模型的已带上下文信息的补丁表示，进而通过自回归方式预测下一个补丁。

2.2. 组件

补丁嵌入器通过 P 的块大小，将字节序列 $x_{0..T}$ 映射为长度为 $K = \frac{T}{P}$ 、特征维度为 $P \cdot D_G$ 的一组补丁嵌入序列。

首先，每个字节都会通过查找表 $E^{\text{global-embed}} \in \mathbb{R}^{V \times D_G}$ 进行嵌入操作，生成大小为 D_G 的嵌入向量，随后还会添加位置嵌入。

$$h_t^{\text{embed}} = E_{x_t}^{\text{global-embed}} + E_t^{\text{pos}} \quad t \in [0..T] \quad (1)$$

接着，字节级嵌入会被重塑为一系列维度为 $P \cdot D_G$ 的 K 补丁更新。为了实现自回归建模，该补丁序列会通过并在开头添加一个可训练的、与补丁大小相同的填充嵌入 ($E^{\text{global-pad}} \in \mathbb{R}^{P \times D_G}$) 来进行填充处理，同时还会移除输入中的最后一个补丁。这个处理后的序列便成为全局模型的输入，记作 $h^{\text{global-in}} \in \mathbb{R}^{K \times (P \cdot D_G)}$ 。

$$h_k^{\text{global-in}} = \begin{cases} E^{\text{global-pad}}, & \text{if } k = 0, \\ h_{((k-1) \cdot P):(k \cdot P)}^{\text{embed}}, & k \in [1, \dots, K), \end{cases} \quad (2)$$

全局模型是一种仅包含解码器的Transformer架构，其 $P \cdot D_G$ 特征维度旨在处理由 K 个补丁更新构成的序列。该模型内置了自注意力机制与因果掩码技术，用以捕捉不同补丁之间的依赖关系。它以 K 个补丁表示的序列 $h_{0:K}^{\text{global-in}}$ ：作为输入，通过针对先前补丁应用自注意力机制，输出经过更新的表示 $h_{0:K}^{\text{global-out}}$ ：。

$$h_{0:K}^{\text{global-out}} = \text{transformer}^{\text{global}}(h_{0:K}^{\text{global-in}}) \quad (3)$$

最终的全局层 $h_{0:K}^{\text{global}}$ 的输出包含 K 个维度为 $P \cdot D_G$ 的补丁表示。对于每一个这样的补丁表示，我们都会将其重塑为长度为 P 、维度为 D_G 的序列，其中位置 p 会使用维度 $p \cdot D_G$ 来 $(p+1) \cdot D_G$ 。随后，每个位置都会通过矩阵 $w^{\text{GL}} \in \mathbb{R}^{D_G \times D_L}$ 被投影到局部模型的维度上，此处 D_L 即为局部模型的维度。之后，我们会将这些投影结果与下一个补丁 $E_{x_{(k \cdot P + p - 1)}}^{\text{local-embed}}$ 中各标记对应的大小为 D_L 的字节嵌入相结合。而局部字节嵌入则会通过一个可训练的局部填充嵌入 ($E^{\text{local-pad}} \in \mathbb{R}^{D_L}$) 向后偏移一个位置。

$$\begin{aligned}
 h_t^{\text{embed}} &= E_{x_t}^{\text{global-embed}} + E_t^{\text{pos}} & t \in [0..T), E^{\text{global-embed}} \in \mathbb{R}^{V \times D_G}, \\
 & & E^{\text{pos}} \in \mathbb{R}^{T \times D_G}, h^{\text{embed}} \in \mathbb{R}^{T \times D_G} \\
 h_k^{\text{global-in}} &= \begin{cases} E^{\text{global-pad}}, & \text{if } k = 0, \\ h_{((k-1) \cdot P):(k \cdot P)}^{\text{embed}}, & k \in [1, \dots, K), \end{cases} & E^{\text{global-pad}} \in \mathbb{R}^{P \times D_G}, K = \frac{T}{P} \\
 h_{0:K}^{\text{global-out}} &= \text{transformer}^{\text{global}}(h_{0:K}^{\text{global-in}}) & h^{\text{global-out}} \in \mathbb{R}^{K \times P \cdot D_G}, h^{\text{global-in}} \in \mathbb{R}^{K \times P \cdot D_G} \\
 h_{k,p}^{\text{local-in}} &= w^{\text{GL}} h_{k,(p \cdot D_G):(p+1) \cdot D_G}^{\text{global-out}} + \begin{cases} E^{\text{local-pad}}, & \text{if } p = 0 \\ E_{x_{(k \cdot P + p - 1)}}^{\text{local-embed}}, & p \in [1, \dots, P) \end{cases} & E^{\text{local-pad}} \in \mathbb{R}^{D_L}, w^{\text{GL}} \in \mathbb{R}^{D_G \times D_L} \\
 & & E^{\text{local-embed}} \in \mathbb{R}^{V \times D_L} \\
 h_{k,0:P}^{\text{local-out}} &= \text{transformer}^{\text{local}}(h_{k,0:P}^{\text{local-in}}) & h_{k,p}^{\text{local-in}} \in \mathbb{R}^{D_L}, h_{k,p}^{\text{local-out}} \in \mathbb{R}^{K \times P \cdot D_L} \\
 p(x_t | x_{0:t}) &= \text{softmax}(E^{\text{local-embed}} h_{k,p}^{\text{local-out}})_{x_t} & t = k \cdot P + p
 \end{aligned}$$

Figure 2. Summary of MEGABYTE with vocabulary V , sequence length T , global and local dimensions D_G and D_L , and K patches of size P . Transformer layers use masked self attention to not observe information from future timesteps.

to allow autoregressive modelling within a patch. This results in a tensor $h^{\text{local-in}} \in \mathbb{R}^{K \times P \times D_L}$.

$$h_{k,p}^{\text{local-in}} = w^{\text{GL}} h_{k,(p \cdot D_G):(p+1) \cdot D_G}^{\text{global-out}} + E_{x_{(k \cdot P + p - 1)}}^{\text{local-embed}} \quad (4)$$

Local Model is a smaller decoder-only Transformer of dimension D_L that operates on a single patch k containing P elements, each of which is the sum of an output from the global model and an embedding of the previous byte in the sequence. K copies of the local models are run on each patch independently (and in parallel during training), computing a representation $h^{\text{local-out}} \in \mathbb{R}^{K \times P \cdot D_L}$.

$$h_{k,0:P}^{\text{local-out}} = \text{transformer}^{\text{local}}(h_{k,0:P}^{\text{local-in}}) \quad (5)$$

Finally, we can compute the probability distribution over the vocabulary at each position. The p th element of the k th patch corresponds to element t of the complete sequence, where $t = k \cdot P + p$:

$$p(x_t | x_{0:t}) = \text{softmax}(E^{\text{local-embed}} h_{k,p}^{\text{local-out}})_{x_t} \quad (6)$$

2.3. Variations and Extensions

We experiment with several extensions of MEGABYTE.

2.3.1. CONVOLUTIONAL PATCH ENCODER

One limitation of chunking sequences into patches is that it is not translation invariant, and byte sequences may receive a different representation depending on their position in the patch. This may mean, for example, that a model has to relearn the meaning of a word at different offsets. To mitigate this issue, we experimented with augmenting the Patch Embedder with causal convolutional layers, which allow translation-invariant contextual representations of the

bytes before they are chunked into patches. We use a stack of convolutional layers, with filter sizes of 3, 5 and 7.

2.3.2. CROSS-PATCH ATTENTION

The Local model uses short sequences for efficiency, and relies on the Global model for long-range information. However, we can increase the context of the Local model with little overhead by allowing it to condition on r elements from the previous patch. This approach allows the Global model to focus on a longer-range context. Specifically, when computing self-attention in each layer, we concatenate the keys and values with the last r keys and queries from the previous patch. We use rotary embeddings (Su et al., 2021) to model relative positions between elements in the sequence. This approach is reminiscent of TransformerXL (Dai et al., 2019) but differs by being fully differentiable.

2.3.3. STRIDED INFERENCE

We observed empirically that the per-token loss within each patch would increase towards the end of the patch, as the prediction relies more on the weaker Local model. To alleviate this issue, we propose *strided inference*, in which we predict the sequence with two forward passes of the full model, whose inputs are offset by $p/2$ positions from each other. We then combine the first $p/2$ positions in each patch for our predictions to predict the complete sequence. Similarly to sliding window techniques (Press et al., 2020), this approach doubles the cost of inference but improves results.

2.4. Motivation

Having described the model, we briefly discuss the motivation behind some of the architectural choices.

Why is the local model needed? Many of the efficiency advantages of the MEGABYTE design could be realized

$$\begin{aligned}
 h_t^{\text{embed}} &= E_{x_t}^{\text{global-embed}} + E_t^{\text{pos}} & t \in [0..T) \quad E^{\text{global-embed}} \in \mathbb{R}^{V \times D_G}, \\
 & & E^{\text{pos}} \in \mathbb{R}^{T \times D_G}, h^{\text{embed}} \in \mathbb{R}^{T \times D_G} \\
 h_k^{\text{global-in}} &= \begin{cases} E^{\text{global-pad}}, & \text{if } k = 0, \\ h_{((k-1) \cdot P):(k \cdot P)}^{\text{embed}}, & k \in [1, \dots, K), \end{cases} & E^{\text{global-pad}} \in \mathbb{R}^{P \times D_G}, K = \frac{T}{P} \\
 h_{0:K}^{\text{global-out}} &= \text{Transformer}^{\text{global}}(h_{0:K}^{\text{global-in}}) & h^{\text{global-out}} \in \mathbb{R}^{K \times P \cdot D_G}, h^{\text{global-in}} \in \mathbb{R}^{K \times P \cdot D_G} \\
 h_{k,p}^{\text{local-in}} &= w^{\text{GL}} h_{k,p \cdot D_G:(p+1) \cdot D_G}^{\text{global-out}} + \begin{cases} E^{\text{local-pad}}, & \text{如果 } p = 0 \\ E_{x_{(k \cdot P + p - 1)}}^{\text{local-embed}}, & p \in [1, \dots, P) \end{cases} & E^{\text{local-pad}} \in \mathbb{R}^{D_L}, w^{\text{GL}} \in \mathbb{R}^{D_G \times D_L} \\
 & & E^{\text{local-embed}} \in \mathbb{R}^{V \times D_L} \\
 h_{k,0:P}^{\text{local-out}} &= \text{Transformer}^{\text{local}}(h_{k,0:P}^{\text{local-in}}) & h_{k,p}^{\text{local-in}} \in \mathbb{R}^{D_L}, h_{k,p}^{\text{local-out}} \in \mathbb{R}^{K \times P \cdot D_L} \\
 p(x_t | x_{0:t}) &= \text{softmax}(E^{\text{local-embed}} h_{k,p}^{\text{local-out}})_{x_t} & t = k \cdot P + p
 \end{aligned}$$

图2。MEGA概要 具有词汇量 V 、序列长度 T 、全局与局部维度 $\{3\}$ 和 $\{4\}$ ，以及 $\{5\}$ 个补丁更新的BYTE模型 f 尺寸为 P 。Transformer架构的层数 通过掩码机制对自注意力机制进行约束，从而避免获取未来时间步的信息。

这样便能够在单个补丁范围内实现自回归建模，最终得到一个形状为 $h^{\text{local-in}} \in \mathbb{R}^{K \times P \times D_L}$ 的张量。

$$h_{k,p}^{\text{local-in}} = w^{\text{GL}} h_{k,(p \cdot D_G):(p+1) \cdot D_G}^{\text{global-out}} + E_{x_{(k \cdot P + p - 1)}}^{\text{local-embed}} \quad (4)$$

局部模型是一种规模较小的仅解码器的Transformer架构，其**特征维度**为 D_L ，它会对包含 P 个元素的单一补丁 k 进行处理，其中每个元素都是全局模型的输出与序列中前一个字节的嵌入值之和。会在每个补丁上独立地（并在训练期间并行地）运行 K 个这样的局部模型，以此计算出相应的表示 $h^{\text{local-out}} \in \mathbb{R}^{K \times P \cdot D_L}$ 。

$$h_{k,0:P}^{\text{local-out}} = \text{transformer}^{\text{local}}(h_{k,0:P}^{\text{local-in}}) \quad (5)$$

最后，我们可以计算每个位置上词汇表上的概率分布。第 k 个补丁的 p 个元素对应于完整序列中的 t 个元素，其中 $t = k \cdot P + p$:

$$p(x_t | x_{0:t}) = \text{softmax}(E^{\text{local-embed}} h_{k,p}^{\text{local-out}})_{x_t} \quad (6)$$

2.3. 变体与扩展

我们对MEGABYTE模型的几种扩展方案进行了实验。

2.3.1. 卷积式补丁编码器

将序列分割为补丁的一个局限性在于其不具备平移不变性，不同位置的字节序列可能会获得不同的表示形式。这意味着模型可能需要针对不同偏移位置重新学习某个单词的含义。为了解决这一问题，我们尝试在补丁嵌入器中加入因果卷积层，以此生成具有平移不变性的上下文表示。

我们在将这些字节分割成补丁之前，会使用由滤波器尺寸为3、5和7的卷积层构成的堆叠结构来处理它们。

2.3.2. 交叉块注意力机制

The Local model uses short sequences for efficiency, and relies on the Global model for long-range information. However, we can increase the context of the Local model with little overhead by allowing it to condition on r elements from the previous patch. This approach allows the Global model to focus on a longer-range context. Specifically, when computing self-attention in each layer, we concatenate the keys and values with the last r keys and queries from the previous patch. We use rotary embeddings (Su et al., 2021) to model relative positions between elements in the sequence. This approach is reminiscent of TransformerXL (Dai et al., 2019) but differs by being fully differentiable.

2.3.3. 步进推理

通过实证观察，我们发现每个补丁内的每个标记对应的损失会随着靠近补丁末尾而增大，这是因为预测过程越来越依赖性能较弱的局部模型。为了解决这一问题，我们提出了步进推理方法，即通过全模型的两次前向传播来预测序列，这两次传播的输入彼此之间相隔 $p/2$ 个位置。随后，我们会结合每个补丁中前 $p/2$ 个位置的标记来进行预测，从而得出整个序列的结果。与滑动窗口技术类似（Press等人，2020年），这种方法的推理成本虽然翻倍，但能提升预测效果。

2.4. 动机分析

在介绍了该模型之后，本文将简要探讨某些架构设计选择背后的动机。

为什么需要局部模型？ MEGABYTE模型的诸多效率优势都可以通过这种方式实现。

with the Global model alone, which would resemble a decoder version of ViT (Dosovitskiy et al., 2020). However, the joint distribution over the patch $p(x_{t+1}, \dots, x_{t+P}|x_{0..t})$ has an output space of size 256^P so direct modeling is only tractable for very small patches. We could instead factor the joint distribution into conditionally independent distributions $p(x_{t+1}|x_{0..t}) \cdot p(x_{t+P}|x_{0..t})$, but this would greatly limit the model’s expressive power. For example, it would be unable to express a patch distribution such as 50% *cat* and 50% *dog*, and would instead have to assign probability mass to strings such as *cag* and *dot*. Instead, our autoregressive Local model conditions on previous characters within the patch, allowing it to only assign probability to the desired strings.

Increasing Parameters for Fixed Compute Transformer models have shown consistent improvements with parameter counts (Kaplan et al., 2020). However, the size of models is limited by their increasing computational cost. MEGABYTE allows larger models for the same cost, both by making self attention sub-quadratic, and by using large feedforward layers across patches rather than individual tokens.

Re-use of Established Components MEGABYTE consists of two transformer models interleaved with shifting, re-shaping and a linear projection. This re-use increases the likelihood that the architecture will inherit the desirable scaling properties of transformers.

3. Efficiency Analysis

3.1. Training Efficiency

We analyze the cost of different architectures when scaling both the sequence length and size of the models.

Attention The cost of the self attention in a transformer architecture for a sequence of length T has $O(T^2)$ complexity. Much work has been explored reducing this; for example, Sparse Transformers (Child et al., 2019) and Routing Transformers (Roy et al., 2020) show strong results with a complexity $O(T^{\frac{3}{2}})$. Numerous linear attention mechanisms have also been proposed (Katharopoulos et al., 2020; Schlag et al., 2021; Choromanski et al., 2020), although we are not aware of competitive results on large scale language modeling tasks. As a function of sequence length T and patch size P , the Global model has a sequence of length $\frac{T}{P}$ so uses $O(\frac{T^2}{P^2})$ operations, and the Local model uses $\frac{P}{T}$ sequences of length P so uses $O(\frac{TP^2}{P}) = O(PT)$ operations. The overall cost of MEGABYTE is therefore in $O(\frac{T^2}{P^2} + TP)$. P is a hyperparameter that is chosen to create an architecture for sequences of size T . By setting $P = T^{\frac{1}{3}}$ the complexity is in $O(T^{\frac{4}{3}})$. Using much shorter patches of $P = T^{\frac{1}{5}}$ would give a complexity of $O(T^{\frac{8}{5}})$. The cost is less than the transformer for all non-trivial values of P such

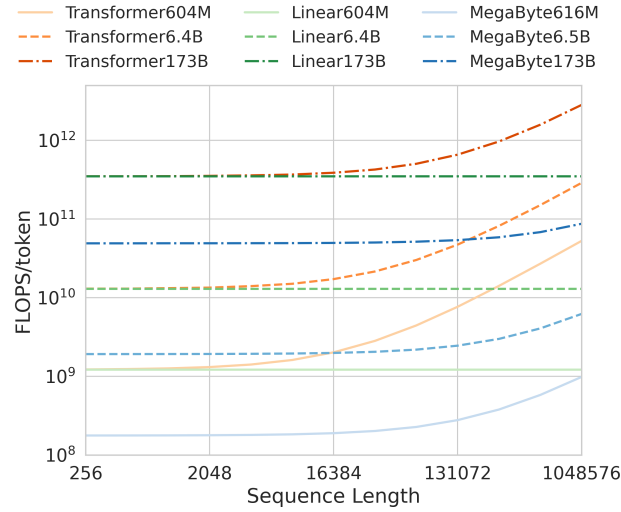


Figure 3. Computational cost (FLOPS/token) for different model architectures at different scales. MEGABYTE architectures (here with $P = 8$) use less FLOPS than equivalently sized Transformers and Linear Transformers (Katharopoulos et al., 2020) across a wide range of model sizes and sequence lengths, allowing larger models to be used for the same computational cost.

that $1 < P < T$.

Feedforward Layers However, attention is not the main cost in large transformers. Instead of increasing the sequence length, transformers are more commonly scaled by increasing the dimension of their latent state d , and the feed-forward network cost dominates the model’s overall cost (Kaplan et al., 2020). For example, in the GPT3 architecture, the quadratic self-attention computation accounts for only 1.4% of FLOPS. Following the approximation of (Kaplan et al., 2020), a forward pass with a large transformer with m non-embedding parameters on a sequence of length T uses roughly $2mT$ FLOPS. MEGABYTE contains two transformers: the Global model uses m_g parameters on a sequence of length $\frac{T}{P}$, and a Local model with m_l parameters that sees $\frac{T}{P}$ sequences of length P , giving an estimate of $2T(\frac{m_g}{P} + m_l)$ FLOPS. When $m_g \gg m_l$, the FLOPS used by MEGABYTE is approximately $\frac{2Tm_g}{P}$, allowing a model P times larger than a transformer with equivalent FLOPS. This analysis holds irrespective of any efficient attention mechanisms used in the transformer.

Combined Analysis To understand efficiency at different sequence lengths and model sizes, we calculate the total FLOPS used by transformers, Linear Transformers and MEGABYTE. For each operation, we use FLOP estimates from (Kaplan et al., 2020), except for attention in Linear Transformers, which we estimate as $9D$ FLOPS/-

with the Global model alone, which would resemble a decoder version of ViT (Dosovitskiy et al., 2020). However, the joint distribution over the patch $p(x_{t+1}, \dots, x_{t+P}|x_{0..t})$ has an output space of size 256^P so direct modeling is only tractable for very small patches. We could instead factor the joint distribution into conditionally independent distributions $p(x_{t+1}|x_{0..t}) \cdot p(x_{t+P}|x_{0..t})$, but this would greatly limit the model’s expressive power. For example, it would be unable to express a patch distribution such as 50% *cat* and 50% *dog*, and would instead have to assign probability mass to strings such as *cag* and *dot*. Instead, our autoregressive Local model conditions on previous characters within the patch, allowing it to only assign probability to the desired strings.

在固定计算资源下增加参数数量 Transformer架构模型的性能会随着参数数量的增加而持续提升 (Kaplan等人, 2020年)。然而，模型规模却受到不断上升的计算成本的限制。MEGABYTE模型通过将自注意力机制的复杂度降为二次方级以下，以及采用跨多个标记而非单个标记的大规模前馈层，使得在相同的成本下能够构建更大的模型。

现有组件的复用 MEGABYTE模型由两个 Transformer模型相互交织而成，其间还穿插着移位、重整形以及线性投影操作。这种组件复用的方式提升了该架构继承Transformer模型优异的缩放特性的可能性。

3. 效率分析

3.1. 训练效率

我们分析了在同时扩展序列长度与模型规模时，不同架构所对应的成本情况。

Attention The cost of the self attention in a transformer architecture for a sequence of length T has $O(T^2)$ complexity. Much work has been explored reducing this; for example, Sparse Transformers (Child et al., 2019) and Routing Transformers (Roy et al., 2020) show strong results with a complexity $O(T^{\frac{3}{2}})$. Numerous linear attention mechanisms have also been proposed (Katharopoulos et al., 2020; Schlag et al., 2021; Choromanski et al., 2020), although we are not aware of competitive results on large scale language modeling tasks. As a function of sequence length T and patch size P , the Global model has a sequence of length $\frac{T}{P}$ so uses $O(\frac{T^2}{P^2})$ operations, and the Local model uses $\frac{P}{T}$ sequences of length P so uses $O(\frac{TP^2}{P}) = O(PT)$ operations. The overall cost of MEGABYTE is therefore in $O(\frac{T^2}{P^2} + TP)$. P is a hyperparameter that is chosen to create an architecture for sequences of size T . By setting $P = T^{\frac{1}{3}}$ the complexity is in $O(T^{\frac{4}{3}})$. Using much shorter patches of $P = T^{\frac{1}{5}}$ would give a complexity of $O(T^{\frac{8}{5}})$. The cost is less than the transformer for all non-trivial values of P such

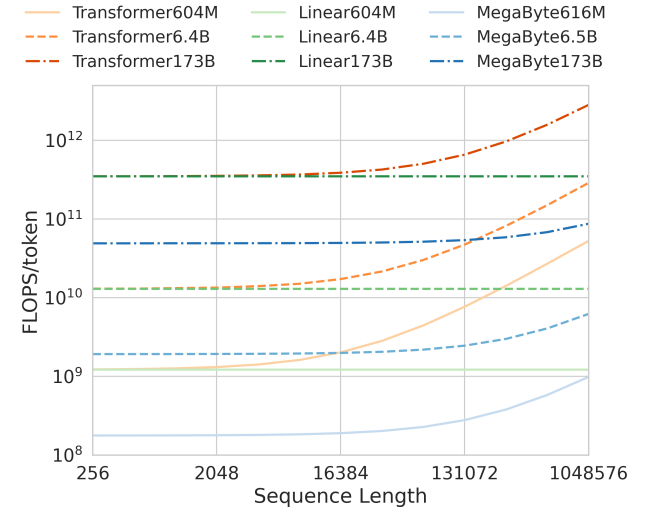


图3.不同规模下各类模型架构的计算成本（浮点运算次数/标记）。MEGABYTE 这类架构（此处用 $P = 8$ ）在各种模型规模和序列长度下，其所需的浮点运算次数都低于同等规模的Transformer架构和线性Transformer模型 (Katharopoulos等人, 2020)，因此即便需要处理更复杂的任务，也能以相同的计算成本使用规模更大的模型。

即 $1 < P < T$ 。

Feedforward Layers However, attention is not the main cost in large transformers. Instead of increasing the sequence length, transformers are more commonly scaled by increasing the dimension of their latent state d , and the feed-forward network cost dominates the model’s overall cost (Kaplan et al., 2020). For example, in the GPT3 architecture, the quadratic self-attention computation accounts for only 1.4% of FLOPS. Following the approximation of (Kaplan et al., 2020), a forward pass with a large transformer with m non-embedding parameters on a sequence of length T uses roughly $2mT$ FLOPS. MEGABYTE contains two transformers: the Global model uses m_g parameters on a sequence of length $\frac{T}{P}$, and a Local model with m_l parameters that sees $\frac{T}{P}$ sequences of length P , giving an estimate of $2T(\frac{m_g}{P} + m_l)$ FLOPS. When $m_g \gg m_l$, the FLOPS used by MEGABYTE is approximately $\frac{2Tm_g}{P}$, allowing a model P times larger than a transformer with equivalent FLOPS. This analysis holds irrespective of any efficient attention mechanisms used in the transformer.

Combined Analysis To understand efficiency at different sequence lengths and model sizes, we calculate the total FLOPS used by transformers, Linear Transformers and MEGABYTE. For each operation, we use FLOP estimates from (Kaplan et al., 2020), except for attention in Linear Transformers, which we estimate as $9D$ FLOPS/-

token¹, where D is the model embedding dimension. Figure 3 shows that for models of size 660M to 173B and sequence lengths of up to 1M tokens, MEGABYTE with $P = 8$ uses less FLOPS than either transformers or Linear Transformers. Baseline model architectures are based on GPT3, and Megabyte global/local model sizes are 452M/151M, 5.8B/604M, 170B/3.2B respectively.

3.2. Generation Efficiency

Generating long sequences with transformers is slow, because the input to each timestep is the output from the previous timestep, meaning each layer must be computed for each token serially. As running a layer on a single token typically does not saturate the amount of parallelism available within a GPU, for analysis, we model each layer as a constant cost independently of size. Consider a MEGABYTE model with L_{global} layers in the Global model and L_{local} layers in the Local model and patch size P , compared with a Transformer architecture with $L_{\text{local}} + L_{\text{global}}$ layers. Generating each patch with MEGABYTE requires a sequence of $O(L_{\text{global}} + P \cdot L_{\text{local}})$ serial operations, whereas the Transformer requires $O(P \cdot L_{\text{global}} + P \cdot L_{\text{local}})$ serial operations. When $L_{\text{global}} \gg L_{\text{local}}$ (i.e. the Global model has many more layers than the Local model), MEGABYTE can reduce inference costs by a factor close to P .

4. Experimental setup

4.1. Controlling for Compute and Data

Models show consistent improvements when increasing both data and compute (Kaplan et al., 2020; Hoffmann et al., 2022), meaning that one model can outperform another because of an increased training budget instead of an improved architecture. However, in practice, both compute and data are typically limited. We conduct experiments using a fixed compute and data budget across all models to focus comparisons solely on the model architecture rather than training resources. To achieve this, we adjust model hyperparameters (mainly, number of layers) within each architecture so that the forward pass time taken per byte is matched, and then train all models for the same number of bytes.

4.2. Comparison Systems

We compare MEGABYTE with both a standard decoder-only Transformer and PerceiverAR (Hawthorne et al., 2022). PerceiverAR extends the original transformer with a single cross-attention layer over a much longer context sequence, and is the best performing general purpose autoregressive model we are aware of and achieves state-of-the-art results

¹This may underestimate the time taken by Linear Transformer decoders, which use a recurrence mechanism that is harder to parallelize on current hardware.

Dataset	Total Bytes	Mean document size (bytes)
PG-19	10.1GB	411,404
Stories	21.3GB	35,265
Books	79.7GB	509,526
arXiv	91.5GB	58,518
Code	353.7GB	7,461

Table 1. Text dataset sizes and mean document lengths.

across several modalities. We implemented both models in the same codebase, and all models share a similar data loader, preprocessing step, and trainer to avoid any artifacts in our compute-controlled experiments.

4.3. Training Procedure

All models were trained using the Metaseq² code base (Zhang et al., 2022b). The training used the PyTorch framework (Paszke et al., 2019), with fairscale to improve memory efficiency through fully sharded model and optimizer states (Baines et al., 2021). Mixed precision training was used to improve training efficiency at scale (Micikevicius et al., 2017). More training details and various model parameters can be found in Section A.1 in the Appendix.

To validate our implementation of PerceiverAR, we reproduced their experiments on downsized ImageNet at 64 pixels. By carefully matching hyperparameters, we achieved a bits per byte (bpb) score of 3.53, compared to the reported 3.54 in the original paper.

4.4. Inference Methods

Several techniques have been proposed for trading off speed for performance during inference with language models, including sliding windows (Press et al., 2020) and our strided inference (Section 2.3.3). We only use these methods when comparing with prior published work (Tables 3 and 4).

5. Language Modeling

We evaluated the performance of MEGABYTE on language modeling on a set of 5 diverse datasets emphasizing long-range dependencies: Project Gutenberg (PG-19), Books, Stories, arXiv, and Code.

Datasets We experiment on a range of long form text datasets. The PG-19 dataset (Rae et al., 2019b) consists of English-language books written before 1919 and is extracted from the Project Gutenberg online library. The Stories dataset (Trinh & Le, 2018) is a subset of CommonCrawl data meant to emulate Winograd schemas. Books (Gao et al., 2020) is another collection of English-language books. The arXiv dataset is a collection of technical publications written

²<https://github.com/facebookresearch/metaseq>

标记¹, 其中 D 表示模型的嵌入维度。图3 显示，在参数量为660M至173B、序列长度高达1M个标记的模型中，配备 $P = 8$ 的MEGABYTE模型相比Transformer模型或线性Transformer模型所需的浮点运算次数更少。基准模型架构基于GPT3构建，而MEGABYTE的全球模型/局部模型参数量分别为452M/151M、5.8B/604M、170B/3.2B。

3.2. 生成效率

使用Transformer架构生成长序列的速度较慢，因为每个时间步的输入都是前一个时间步的输出，这意味着必须逐个顺序地为每个标记计算每一层。由于对单个标记处理一层通常不会耗尽图形处理器内的并行处理能力，因此在分析中我们将每层的计算成本视为与层大小无关的固定值。以全局模型中含有 L_{global} 层、局部模型中含有 L_{local} 层且块大小为 P 的MEGABYTE模型，与含有 $L_{\text{local}} + L_{\text{global}}$ 层的Transformer架构进行比较：使用MEGABYTE模型生成每个块需要 $O(L_{\text{global}} + P \cdot L_{\text{local}})$ 次串行操作，而Transformer架构则需要 $O(P \cdot L_{\text{global}} + P \cdot L_{\text{local}})$ 次串行操作。当 $L_{\text{global}} \gg L_{\text{local}}$ （即全局模型的层数远多于局部模型）时，MEGABYTE模型能够将推理成本降低接近 P 倍。

4. 实验环境配置

4.1. 控制计算与数据因素

当数据量和计算资源同时增加时，模型性能会持续提升（Kaplan等人，2020年；Hoffmann等人，2022年），这说明某个模型之所以能超越另一个模型，往往是由于训练预算的增加而非模型架构的改进。然而在实践中，计算资源和数据量通常都是有限的。为了将比较重点完全放在模型架构上而非训练资源上，我们在所有模型中都采用固定的计算和数据预算来进行实验。为此，我们会调整每种架构下的模型超参数（主要是层数），使得每字节的前向传播时间保持一致，然后再让所有模型处理相同字节数的数据进行训练。

4.2. 对比系统

我们对比了MEGABYTE 与传统的仅解码器的Transformer架构以及PerceiverAR(Hawthorne等人，2022年)的表现。PerceiverAR在原始Transformer架构的基础上增加了一层跨注意力层，能够处理更长的上下文序列；据我们所知，它是目前性能最佳的通用自回归模型，并且取得了行业顶尖的成果。

¹这种估算方式可能会低估Linear Transformer解码器所需的时间，因为这类解码器采用了递归机制，在当前的硬件环境下较难实现并行处理。

数据集	总字节数	平均文档大小（字节）
PG-19数据集	10.1GB	411,404
故事数据集	21.3GB	35,265
书籍数据集	79.7GB	509,526
arXiv数据库	91.5GB	58,518
Code	353.7GB	7,461

表1。文本数据集的规模与文档平均长度。

这些实验涵盖了多种不同的数据模态。我们在同一个代码库中实现了这两种模型，而且所有模型都使用了相似的数据加载器、预处理步骤以及训练器，以此确保在受计算条件控制的实验中不会出现任何干扰因素。

4.3. 训练流程

All models were trained using the Metaseq² code base (Zhang et al., 2022b). The training used the PyTorch framework (Paszke et al., 2019), with fairscale to improve memory efficiency through fully sharded model and optimizer states (Baines et al., 2021). Mixed precision training was used to improve training efficiency at scale (Micikevicius et al., 2017). More training details and various model parameters can be found in Section A.1 in the Appendix.

为验证我们对PerceiverAR模型的实现效果，我们在尺寸缩小至64像素的ImageNet数据集上重现了他们的相关实验。通过精心匹配各种超参数，我们最终获得的每字节比特数（bpb）值为3.53，与原始论文中报告的3.54数值基本一致。

4.4. 推理方法

在语言模型推理过程中，为在速度与性能之间取得平衡，人们提出了多种技术，包括滑动窗口法（Press等人，2020年）以及我们提出的步进推理方法（见2.3.3节）。只有在与以往已发表的研究进行对比时，我们才会使用这些方法（见表3 和4）。

5. 语言建模

我们在包含5个类型各异、注重长距离依赖关系的数据集上，评估了MEGABYTE模型在语言建模任务中的性能，这些数据集分别是：古腾堡计划（PG-19）数据集、书籍数据集、故事数据集、arXiv数据库以及代码数据集。

数据集我们在多种长文本形式的数据集上进行了实验。PG-19数据集（Rae等人，2019b）包含1919年之前出版的英文书籍，这些数据是从古腾堡计划在线图书馆中提取的。Stories数据集（Trinh与Le，2018）是CommonCrawl数据集的一个子集，旨在模拟Winograd模式。Books数据集（Gao等人，2020）则是另一组英文书籍的集合。arXiv数据集则是由各类技术类出版物构成的集合

²<https://github.com/facebookresearch/metaseq>

MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers					
	PG-19	Stories	Books	arXiv	Code
Transformer	1.057	1.064	1.097	0.816	0.575
PerceiverAR	1.104	1.070	1.104	0.791	0.546
MEGABYTE	1.000	0.978	1.007	0.678	0.411

Table 2. Performance (bits-per-byte) of compute and data controlled MEGABYTE, PerceiverAR, and Transformer models on various text modalities.

in L^AT_EX from the [arXiv online archive](#). Finally, the Code dataset is a large publicly available dataset of open source code, under Apache, BSD or MIT licenses. More details on dataset sizes and document lengths are shared in Table 1.

Controlled Experiments Table 2, lists bpb on each dataset. Each model is trained for 80 billion bytes, and models are scaled to use the same compute budget. We carefully tune hyperparameters for all architectures to best utilize the available compute budget. MEGABYTE consistently outperforms both baseline transformers and PerceiverAR across all datasets. We use the same set of parameters on all dataset. In all experiments presented in Table 2, transformer has size of 320M with context length of 1024, PerceiverAR has size of 248M with context size of 8192 and latent size of 1024, and MEGABYTE global/local model sizes are 758M/262M with context length of 8192 and patch size of 8.

Scaling Experiment We scale up our training data on PG-19 (Table 3), and compare MEGABYTE with byte baselines, as well as converting all results to word-level perplexities to benchmark with state-of-art token based models.

We train a byte-level Transformer, PerceiverAR and MEGABYTE models for 400B bytes and the same compute budget using same model parameters as in the controlled experiments. We find that MEGABYTE outperforms other byte-level models by a wide margin at this scale.³

We also compare with the best previously reported numbers for sub-word models. These results may be confounded by differing amounts of compute and tuning used, but show that MEGABYTE gives results competitive with state-of-the-art models trained on subwords. These results suggest that MEGABYTE may allow future large language models to be tokenization-free.

³The only prior byte-level experiments we are aware of are at a smaller scale in [Hutchins et al. \(2022\)](#), who report results equivalent to test perplexities of 46.5 with a version of the Block-Recurrent transformer, and 49.5 with Memorizing Transformers ([Wu et al., 2022](#)), compared to 36.4 with our model.

6. Image Modeling

6.1. Sequence Modeling on ImageNet

We test MEGABYTE on variants of the autoregressive image generation task on ImageNet ([Oord et al., 2016](#)), to measure its ability to efficiently use long context. We test on three different resolutions of images, ranging from 64×64 to 640×640 pixels – the latter requiring the effective modeling of sequences with over 1.2M tokens. This generation task becomes increasingly challenging as the image’s resolution grows: doing well on this task requires the modeling of local patterns (textures, lines, etc.) and long-range context that provides information about the high level structure of the image. Inspired by recent works in Vision Transformers ([Dosovitskiy et al., 2020](#)), we model image data patch by patch (more details can be found in Appendix D.1).

6.2. Comparison with State of the Art

We train a large MEGABYTE model on ImageNet 64x64 with Global and Local models sized 2.7B and 350M parameters, respectively, for 1.4T tokens. We estimate that training this model consumed less than half the GPU hours we would have needed to reproduce the best PerceiverAR model described by ([Hawthorne et al., 2022](#)). As shown in Table 4, MEGABYTE matches the state-of-the-art performance of PerceiverAR whilst using only half the compute.

6.3. Scaling to higher resolutions

We compare three transformer variants (vanilla, PerceiverAR, MEGABYTE) to test scalability to long sequences on increasingly large image resolutions. We use our own implementations of these in the same framework and budget the same amount of GPU hours and data to train each of these model variants.

MEGABYTE is able to handle all sequence lengths with a single forward pass of up to 1.2M tokens. We found neither the standard Transformer nor PerceiverAR could model such long sequences at a reasonable model size, so instead we split images into segments of size 1024 and 12000 respectively. For Megabyte, we set patch size as 12 for Image64 and patch size as 192 for Image256 and Image640 datasets. Model sizes are adjusted to match overall training speeds across models and we do not use any form of sliding window evaluation in this experiment. As seen in Table 5, MEGABYTE outperforms baselines across all resolutions in this compute-controlled setting. The precise settings used for each of the baseline models such as context length and number of latents are summarized in Table 11.

Results show that MEGABYTE outperforms the other systems at all resolutions, demonstrating an effective model of sequences of over 1M bytes.

MEGABYTE模型：利用多尺度变换器预测百万字节级的序列数据					
	PG-19数据集	故事数据集	书籍数据集	arXiv数据库	Code
Transformer架构	1.057	1.064	1.097	0.816	0.575
PerceiverAR模型	1.104	1.070	1.104	0.791	0.546
MEGABYTE模型	1.000	0.978	1.007	0.678	0.411

表2。在不同文本模式下，经过计算与数据控制优化的MEGABYTE模型、PerceiverAR模型以及Transformer架构的性能表现（每字节比特数）。

在层深度方面来自[arXiv在线档案库](#)的文本。最后，Code数据集是一个规模庞大的公开可用数据集，其中包含遵循Apache、BSD或MIT许可证的开源代码。关于数据集大小和文档长度的更多详细信息详见表1中。

对照实验表2列出了各数据集对应的bpb值。所有模型均经过800亿字节的训练，并且会对模型进行缩放处理以确保它们使用相同的计算资源。我们会仔细调整各类架构的超参数，从而最充分地利用现有的计算资源。在所有数据集上，MEGABYTE模型的表现始终优于基准Transformer模型以及PerceiverAR模型。我们在所有数据集上使用的参数设置是完全一致的。在表2中展示的所有实验中，Transformer模型的规模为320M，上下文长度为1024；PerceiverAR模型的规模为248M，上下文长度为8192，潜在空间尺寸也为1024；而MEGABYTE模型的全局模型与局部模型的规模分别为758M和262M，其上下文长度为8192，块大小为8。

扩展实验我们扩大了PG-19数据集的规模（见表3），并将MEGABYTE模型与基于字节级的基准模型进行对比，同时将所有结果转换为词级困惑度，以便与最先进的基于标记的模型进行基准测试。

我们在相同的计算资源预算下，使用与控制实验中相同的模型参数，针对400B字节的数据量训练了字节级Transformer架构、PerceiverAR模型以及MEGABYTE模型。测试结果表明，在这种规模下，MEGABYTE模型的性能远远优于其他字节级模型。³

我们还将这些结果与此前报道的优质子词模型性能进行了对比。尽管不同的计算资源投入和调优方式可能影响了对比结果的准确性，但这些数据仍表明MEGABYTE模型所展现出的性能足以与基于子词训练的先进模型相媲美。这些结果暗示，MEGABYTE或许能让未来的大型语言模型无需进行标记化处理。

³据我们所知，此前仅有规模较小的字节级实验，出自[Hutchins等人 \(2022年\)](#)，他们使用某种版本的Block- Recurrent变换器得到的测试困惑度为46.5，而使用Memorizing变换器（[Wu等人，2022年](#)）得到的测试困惑度为49.5，相比之下，我们的模型对应的测试困惑度为36.4。

6. 图像建模

6.1. 基于ImageNet数据集的序列建模

We test MEGABYTE on variants of the autoregressive image generation task on ImageNet ([Oord et al., 2016](#)), to measure its ability to efficiently use long context. We test on three different resolutions of images, ranging from 64×64 to 640×640 pixels – the latter requiring the effective modeling of sequences with over 1.2M tokens. This generation task becomes increasingly challenging as the image’s resolution grows: doing well on this task requires the modeling of local patterns (textures, lines, etc.) and long-range context that provides information about the high level structure of the image. Inspired by recent works in Vision Transformers ([Dosovitskiy et al., 2020](#)), we model image data patch by patch (more details can be found in Appendix D.1).

6.2. 与现有最优方法的对比

我们在ImageNet 64x64数据集上训练了一个规模庞大的MEGABYTE模型，该模型配备了参数数量分别为27亿参数模型和3.5亿参数模型的全局模型与局部模型，总共处理了1.4万亿个标记。据估计，训练这个模型所消耗的图形处理器使用时间还不到复制由([Hawthorne等人,2022年](#))描述的最佳PerceiverAR模型所需时间的一半。如<table style id='5'>4表所示，MEGABYTE模型在仅使用一半计算资源的情况下，就能达到与PerceiverAR模型相当的最先进性能水平。

6.3. 扩展至更高分辨率

我们对比了三种Transformer架构的变体（普通版、Per-ceiverAR以及MEGABYTE模型），旨在测试其在图像分辨率不断提升的情况下处理长序列数据的扩展能力。我们在同一框架内使用自行实现的版本，并为训练这些不同模型的变体分配了相同数量的图形处理器运行时与数据资源。

MEGABYTE is able to handle all sequence lengths with a single forward pass of up to 1.2M tokens. We found neither the standard Transformer nor PerceiverAR could model such long sequences at a reasonable model size, so instead we split images into segments of size 1024 and 12000 respectively. For Megabyte, we set patch size as 12 for Image64 and patch size as 192 for Image256 and Image640 datasets. Model sizes are adjusted to match overall training speeds across models and we do not use any form of sliding window evaluation in this experiment. As seen in Table 5, MEGABYTE outperforms baselines across all resolutions in this compute-controlled setting. The precise settings used for each of the baseline models such as context length and number of latents are summarized in Table 11.

测试结果显示，在所有分辨率下MEGABYTE模型的表现均优于其他系统，充分证明了其在处理长度超过1M字节的序列数据时具备出色的能力。

MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers					
	Tokenizer	Vocab Size	Context Length	Validation	Test
TransformerXL (Rae et al., 2019a)	SentencePiece	32k	512+1024 (subwords)	45.5	36.3
CompressiveTransformer (Rae et al., 2019a)	SentencePiece	32k	512+512+2x512 (subwords)	43.4	33.6
PerceiverAR (Hawthorne et al., 2022)	SentencePiece	32k	2048 (subwords)	45.9	28.9
BlockRecurrent (Hutchins et al., 2022)	SentencePiece	32k	1024+recurrence (subwords)	-	26.5
Transformer byte-level (ours)	Bytes	256	2048 (bytes)	81.6	69.4
PerceiverAR byte-level (ours)	Bytes	256	8192 (bytes)	119.1	88.8
MEGABYTE	Bytes	256	8192 (bytes)	42.8	36.4

Table 3. Larger scale experiments on PG19, converting bits-per-byte to word-level perplexities for comparison with prior work. Results below the line are compute-matched. MEGABYTE outperforms other byte models by a wide margin, and gives results competitive with state-of-the-art models trained on subwords.

ImageNet64	bpb
Routing Transformer (Roy et al., 2020)	3.43
Combiner (Ren et al., 2021)	3.42
Perceiver AR (Hawthorne et al., 2022)	3.40
MEGABYTE	3.40

Table 4. Bits per byte (bpb) on ImageNet 64×64. MEGABYTE matches the current state-of-the-art while only using half the amount of GPU hours to train.

	Context	Image64	Image256	Image640
Total len		12288	196608	1228800
Transformer	1024	3.62	3.801	2.847
Perceiver AR	12000	3.55	3.373	2.345
MEGABYTE	Full	3.52	3.158	2.282

Table 5. Bits per byte (bpb) on ImageNet with different resolutions. All models use the same compute and data. MEGABYTE scales well to sequences of over 1M tokens.

7. Audio Modeling

Audio has aspects of both the sequential structure of text and the continuous nature of images, so is an interesting application for MEGABYTE.

Raw audio is typically stored as a sequence of 16-bit integer values (one per timestep); a softmax layer would need to output 65,536 probabilities per timestep to model all possible values. To address this issue, various techniques have been developed to reduce the memory and computational requirements of the softmax layer. For instance, van den Oord et al. (2016) apply μ -law companding transformation and quantizes the input into 256 possible values. Alternatively, van den Oord et al. (2017) model the samples using the discretized mixture of logistics distribution introduced by Salimans et al. (2017). Finally, Kalchbrenner et al. (2018) use a dual softmax technique to produce 8 coarse and 8 fine bits. In our approach, we simplify the audio modeling process by directly reading the bytes (256 possible values) from the audio file and conducting an autoregressive language

	Global Size	(Local) Size	bpb	Generation Time (s)
Transformer	-	350M	1.064	132
MEGABYTE	1.3B	218M	0.991	93

Table 6. Comparison of bits per byte (bpb) and generation speed of 8192 bytes of transformer model (with context length 1024) and MEGABYTE with context length 8192 and patch size 8.

model on top of that. This greatly streamlines the modeling process, making it easier and more efficient.

Our audio modeling approach focuses on 16 kHz, 16-bit audio, which equates to 32k bytes per one-second clip. We use an extensive audio dataset consisting of 2 terabytes (roughly 18,000 hours) of audio. We use a sequence length of 524,288, a patch size of 32, and a batch size of 32 to facilitate model training. By utilizing these settings, we can effectively train our model on large volumes of audio data, helping to improve its accuracy and efficacy.

Our model obtains bpb of 3.477, much lower than the results with perceiverAR (3.543) and vanilla transformer model (3.567). More ablation results are presented in Table 7.

8. Analysis

8.1. Generation speed

We also compare the text generation speed between MEGABYTE and a transformer. We compare a 350M parameter baseline transfomer and a MEGABYTE model with a 1.3B parameter Global model and a 218M parameter local model, trained on PG19 with equal compute. As shown in Table 6, the MEGABYTE model achieves much lower perplexity as expected. However, MEGABYTE also generates a sequence of 8192 tokens 40% *faster* than transformer, despite having over 4 times the parameters. This speed up is due to the bulk of the parameters being in the Global model, which only needs to be computed once for every 8 tokens, whereas all the parameters in the baseline model are used on every token.

MEGABYTE模型：利用多尺度变换器预测百万字节级的序列数据					
	分词器	词汇表大小	上下文长度	验证	Test
TransformerXL (Rae等人, 2019a)	SentencePiece算法	32k	512+1024 (子词分解)	45.5	36.3
CompressiveTransformer (Rae等人, 2019a)	SentencePiece算法	32k	512+512+2x512 (子词分解)	43.4	33.6
PerceiverAR模型 (Hawthorne等人, 2022年)	SentencePiece算法	32k	2048 (子词)	45.9	28.9
BlockRecurrent (Hutchins等人, 2022年)	SentencePiece算法	32k	1024+递归结构 (子词)	-	26.5
基于字节的Transformer架构 (我们开发的版本)	字节	256	2048 (字节)	81.6	69.4
PerceiverAR模型的字节级数值 (我们自行设计的版本)	字节	256	8192 (字节)	119.1	88.8
MEGABYTE模型	字节	256	8192 (字节)	42.8	36.4

表3。在PG19数据集上进行的更大规模实验中，将每字节比特数转换为词级困惑度，以便与先前的研究结果进行对比。行以下的数值均为经过计算匹配后的结果。MEGABYTE模型不仅大幅优于其他基于字节的模型，其性能甚至可与那些在子词层面训练的顶尖模型相媲美。

ImageNet64	bpb
Routing Transformer (Roy等人, 2020)	3.43
Combiner (Ren等人, 2021)	3.42
Perceiver AR (Hawthorne等人, 2022年)	3.40
MEGABYTE模型	3.40

表4。在ImageNet 64×64数据集上的每字节比特数 (bpb)。MEGABYTE模型虽仅使用一半的图形处理器运行时间即可完成训练，但其性能仍处于当前最先进水平。

	上下文	Image64分辨率	Image256分辨率	Image640分辨率
总长度		12288	196608	1228800
Transformer架构	1024	3.62	3.801	2.847
Perceiver AR系	12000	3.55	3.373	2.345
统				
MEGABYTE模型	Full	3.52	3.158	2.282

表5。不同分辨率下*ImageNet*上的每字节比特数 (*bpb*)。所有模型均使用相同的计算资源与数据集，而*MEGABYTE*模型在处理超过*100万*个标记的序列时也能表现出良好的扩展性。

7. 音频建模

音频既具有文本的序列结构特征，又具备图像的连续特性，因此是非常适合应用MEGABYTE模型的领域。

Raw audio is typically stored as a sequence of 16-bit integer values (one per timestep); a softmax layer would need to output 65,536 probabilities per timestep to model all possible values. To address this issue, various techniques have been developed to reduce the memory and computational requirements of the softmax layer. For instance, van den Oord et al. (2016) apply μ -law companding transformation and quantizes the input into 256 possible values. Alternatively, van den Oord et al. (2017) model the samples using the discretized mixture of logistics distribution introduced by Salimans et al. (2017). Finally, Kalchbrenner et al. (2018) use a dual softmax technique to produce 8 coarse and 8 fine bits. In our approach, we simplify the audio modeling process by directly reading the bytes (256 possible values) from the audio file and conducting an autoregressive language

	全局 Size	(局部) Size	bpb	生成时间 (秒)
Transformer架构	-	350M	1.064	132
MEGABYTE模型	1.3B	218M	0.991	93

表6。Transformer架构模型（上下文长度为1024）与MEGABYTE模型（上下文长度为8192，块大小为8）在每字节bpb值及生成速度方面的对比。

构建建立在该基础之上的模型。这极大简化了建模流程，使其更加便捷高效。

我们的音频建模方法以16 kHz、16位格式的音频为处理重点，这意味着每秒长度的音频片段对应32k字节的数据量。我们使用了规模庞大的音频数据集，其总容量达2太字节，相当于约18000小时的音频内容。为便于模型训练，我们设定序列长度为524288，块大小为32，批量大小也为32。通过采用这些参数设置，我们能够在大规模的音频数据上高效地训练模型，从而提升其准确度与性能。

我们的模型获得的bpb值为3.477，远低于PerceiverAR模型（3.543）以及普通Transformer架构模型（3.567）的测试结果。更多消融实验的结果请参见表格 7。

8. 分析

8.1. 生成速度

我们还对比了MEGABYTE模型与Transformer架构在文本生成速度方面的表现。具体而言，我们在相同的计算资源条件下，将一个拥有3.5亿参数基准的Transformer模型，以及分别基于PG19数据集训练、拥有13亿参数的全局模型和2.18亿参数的局部模型的MEGABYTE模型进行了测试。如表6所示，正如预期那样，MEGABYTE模型的困惑度要低得多。不过，尽管其参数量是前者4倍多，它生成包含8192个标记的序列的速度仍比Transformer快40%。这种速度提升得益于大部分参数都集中在全局模型中，该模型仅需每8个标记计算一次即可，而基准模型中的所有参数则需要在每个标记上都被使用。

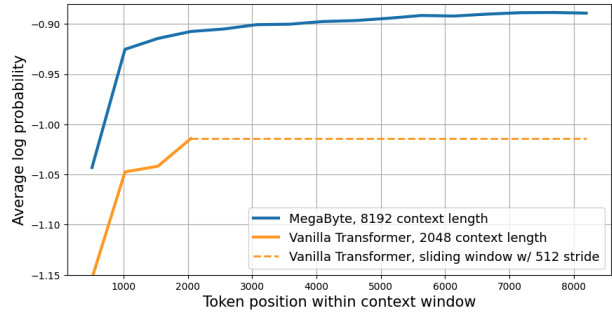


Figure 4. Average log probability assigned to the token at different positions within the context length by MEGABYTE model with 8192 context size and by a vanilla transformer model trained using the same compute (PG19 test set). MEGABYTE likelihoods rise throughout its context window, demonstrating that it can use tokens from 8k bytes previously to improve its predictions.

8.2. Model Components

In Table 7, we analyze the significance of different components in the MEGABYTE architecture by studying arXiv, Librilight-L and ImageNet256 datasets. Removing Local (*w/o local model*) or global (*w/o global model*) model, we observe a substantial increase in bpb on all datasets, showing that both parts are crucial. The performance of the model without the cross-patch local model (*w/o cross-patch local model*) is competitive, indicating that the architecture is robust to this modification. We observe slight improvement on the Librilight-L and ImageNet256 datasets by augmenting the MEGABYTE model with a CNN encoder (*w/ CNN encoder*). This suggests that the MEGABYTE architecture can benefit from integrating alternative encoding mechanisms.

	Arxiv	Audio	ImageNet256
MEGABYTE	0.6871	3.477	3.158
<i>w/o local model</i>	1.263	5.955	4.768
<i>w/o global model</i>	1.373	3.659	3.181
<i>w/o cross-patch attention</i>	0.6781	3.481	3.259
<i>w/ CNN encoder</i>	0.6871	3.475	3.155

Table 7. Ablation of MEGABYTE model components, showing that both Local and Global models are critical to strong performance, but the architecture is robust to other modifications. We report bits-per-byte on text, audio, and image prediction tasks. All models within a column are trained using the same compute and data. The hyperparameters are listed in Table 11.

8.3. Effective Use of Context

Long-context models often struggle to benefit from the full context (Sun et al., 2021). Figure 4 shows that later tokens within each context window consistently have a higher likelihood, indicating that MEGABYTE can effectively use at least 8k bytes of context on the PG19 dataset.

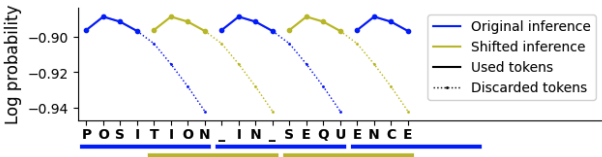


Figure 5. An illustration of strided inference with patch size 8. Lines below the text represent the patches used in the two rounds of inference, the plot above it represents the average probability assigned to the token at a given position within a patch. By considering only the first half of each patch from the two rounds of inference and combining them (bold lines on top), we achieve a better overall bpb.

Method	Inference Cost	bpb
Basic Inference	1X	0.9079
<i>w/ Sliding Window</i>	2X	0.8918
<i>w/ Strided Inference</i>	2X	0.8926
<i>w/ Sliding & Strided</i>	4X	0.8751

Table 8. Performance of various inference techniques on the PG19 test set using our best MEGABYTE model.

8.4. Strided Inference

We find that within a single patch, on average, the MEGABYTE performs worse on later tokens within a patch (see Figure 5). Section 2.3.3 proposes *strided inference* as a solution, where two forward passes are performed offset by $\frac{P}{2}$ tokens, and results from the first half of each patch are combined. Table 8 shows performance improvements from strided inference, which are additive with the standard sliding window.

8.5. Hyperparameters

MEGABYTE introduces several additional hyperparameters. We tuned these parameters independently for different modalities and reported performance based on the best setting we found. All experiments in the same group use the same compute.

Patch Size. We experimented with various patch sizes on Image256 dataset and found that there is a wide range of values where MEGABYTE performs similarly. We found similar robustness against the choice of this hyperparameter across all modalities, although the optimal patch size itself can be different across modalities.

Patch Size	Global Size	Local Size	bpb
48	125M	114M (D=768, L=11)	3.178
192	125M	125M (D=768, L=12)	3.158
768	125M	83M (D=768, L=8)	3.186

Table 9. Effects of patch size on performance on the Image256 dataset. All versions use the same amount of GPU hours and data.

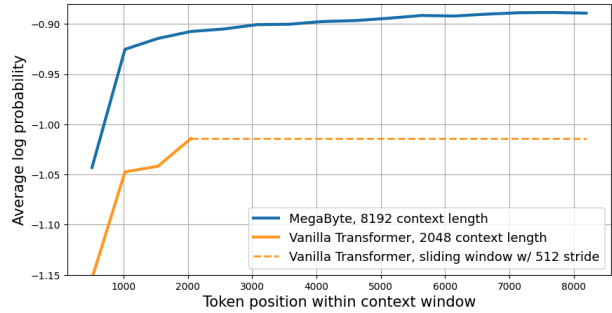


图4。展示的是MEGABYTE模型在8192的上下文长度设定下，以及使用相同计算资源训练的普通Transformer模型在PG19测试集上，对上下文长度内不同位置处的标记所赋予的平均对数概率值。可以看出，MEGABYTE模型的概率值在其整个上下文窗口内持续上升，这表明它能够利用之前8k字节的标记来提升预测精度。

8.2. 模型组件

In Table 7, we analyze the significance of different components in the MEGABYTE architecture by studying arXiv, Librilight-L and ImageNet256 datasets. Removing Local (*w/o local model*) or global (*w/o global model*) model, we observe a substantial increase in bpb on all datasets, showing that both parts are crucial. The performance of the model without the cross-patch local model (*w/o cross-patch local model*) is competitive, indicating that the architecture is robust to this modification. We observe slight improvement on the Librilight-L and ImageNet256 datasets by augmenting the MEGABYTE model with a CNN encoder (*w/ CNN encoder*). This suggests that the MEGABYTE architecture can benefit from integrating alternative encoding mechanisms.

	arXiv	音频	ImageNet256
MEGABYTE模型	0.6871	3.477	3.158
未采用局部模型	1.263	5.955	4.768
未采用全局模型	1.373	3.659	3.181
未使用交叉块注意力机制	0.6781	3.481	3.259
配备了卷积神经网络编码器	0.6871	3.475	3.155

表7。消融实验MEGABYTE 各模型组件的对比结果表明，无论是局部模型还是全局模型都对实现优异的性能至关重要，而该架构对其他类型的修改也具备较强的鲁棒性。我们报告了在文本、音频及图像预测任务中的每字节比特数。同一列中的所有模型均使用相同的计算资源与数据集进行训练，相关超参数则列于表11中。

8.3. 上下文的有效利用

长上下文模型往往难以充分利用全部上下文信息（Sun等人, 2021年）。图4 显示，每个上下文窗口中较后的标记数通常具有更高的出现概率，这表明MEGABYTE模型在处理PG19数据集时能够有效利用至少8k字节的上下文信息。

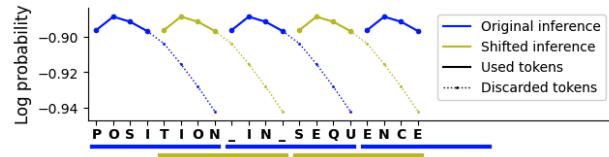


图5。该图展示了采用8像素块大小的步进推理过程。文字下方的线条代表两轮推理中使用的补丁，上方的曲线则表示每个补丁内特定位置上的标记所对应的平均概率。通过仅选取两轮推理中每个补丁的前半部分并将它们合并（即顶部的粗线），我们可以获得更优的总体bpb值。

方法	推理成本	bpb
基础推理	1X	0.9079
带滑动窗口	2X	0.8918
采用步进推理技术	2X	0.8926
结合滑动与步进推理方式	4X	0.8751

表8。使用我们最优的MEGABYTE模型，在PG19测试集上各种推理技术的性能表现。

8.4. 步进推理

我们发现，在单个补丁中，平均而言，MEGABYTE 在处理该补丁中较后的标记时表现更差（见图5）。第2.3.3节提出步进推理作为解决方案，即通过 $\frac{P}{2}$ 标记数的偏移执行两次前向传播，再将每个补丁前半部分的结果进行合并。表8 显示了步进推理所带来的性能提升，这种提升与传统的滑动窗口方法的效果是可叠加的。

8.5. 超参数

MEGABYTE模型引入了多项额外的超参数。我们对不同模态的这些参数分别进行了调优，并以找到的最佳设置对应的性能作为评估依据。同一组中的所有实验均使用相同的计算资源。

块大小。我们在Image256数据集上测试了多种不同的块大小，发现当使用1.25亿参数模型时，其在相当宽的数值范围内都能表现出相似的性能。尽管不同模态下的最优块大小可能有所差异，但该超参数的选择对所有模态而言都具有相近的稳定性。

块大小	全局尺寸	局部尺寸	bpb
48	125M	1.14亿参数模型（特征维度=768，层深度=11）	3.178
192	125M	1.25亿参数模型（特征维度=768，层深度=12）	3.158
768	125M	8300万（特征维度=768，层深度=8）	3.186

表9。Image256数据集上块大小对性能的影响。所有测试版本所使用的图形处理器运行时长及数据量均保持一致。

MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers		
Global Size	Local Size	bpb
350M (D=1024,L=24)	290M (D=1024,L=20)	1.014
760M (D=1536,L=24)	262M (D=1024,L=18)	1.002
1.3B (D=2048,L=24)	218M (D=1024,L=15)	0.991

Table 10. Effects of Local / Global model size on performance on the PG19 dataset. Increasing the capacity of global model improves performance. Models are compute and data matched.

Local to Global model Size Ratio. We experimented with different Local/Global model size ratios on PG19 dataset. By grouping bytes into patches, MEGABYTE effectively uses P times less tokens for the Global model as on the Local model—enabling us to increase the size of the Global model without reduced cost. We find that a given compute budget is spent optimally when the Global model has more parameters than the Local model. This trend was consistent across all modalities and various patch sizes.

9. Related Work

Prior research has explored the possibility of improving the efficiency of Transformers on long sequences, primarily motivated by mitigating the quadratic cost of self-attention.

Efficient Encoder Models Several related techniques to ours have been developed for transformer encoder architectures but cannot be straightforwardly applied to decoders. In particular, patchifying operations have previously been used in image *encoder* models such as ViT (Dosovitskiy et al., 2020), and down- and up-sampling operations have been used for text encoders (Clark et al., 2022), but such methods cannot be naively applied to decoder-only models without leaking information to future bytes in the same patch. MEGABYTE generalizes these approaches to an efficient decoder model by using a intra-patch transformer to predict each sequence element’s likelihood, and offseting the inputs to the two models to avoid leaking information. Jaegle et al. (2021) which uses self-attention on a shorter latent sequence, and Didolkar et al. (2022) which uses recurrent model to process chunks with k input steps also resemble patchification, but this technique cannot easily be applied to decoder architectures without leaking information to future timesteps.

Efficient Decoder models Improving the efficiency of decoder models is more challenging because of the need to make one prediction per timestep, and not leak information to future timesteps. The most popular approaches can be categorized as (1) chunking sequences into smaller blocks, and propagating information from previous blocks with either recurrence (Dai et al., 2019; Hutchins et al., 2022) or cross-attention (Hawthorne et al., 2022), (2) linear alternatives to attention, which typically involve forms of token-level recurrence (Katharopoulos et al., 2020; Schlag et al., 2021)

or state space models (Gu et al., 2021; Smith et al., 2022; Ma et al., 2022), or (3) sparse approximations of attention (Kitaev et al., 2020; Beltagy et al., 2020; Child et al., 2019; Wu et al., 2022). However, the performance of dense attention means it is typically still chosen for large scale decoders (Touvron et al., 2023; Chowdhery et al., 2022). MEGABYTE takes the alternative approach of decomposing the complete sequence into two shorter sequences, giving sub-quadratic attention. We also note that feedforward networks are the dominant cost in large decoders, not self-attention. Our approach to compressing sequences allows much larger models than would be possible when using large feedforward networks at every timestep.

Tokenization The most common approach to shortening sequence lengths in Transformer decoders is to pre-process the input with a form of tokenization, in which multiple bytes are mapped to a single discrete token from a fixed vocabulary. For text, this can be done losslessly using methods such as BPE (Sennrich et al., 2015) and SentencePiece (Kudo & Richardson, 2018), but these approaches can require language-specific heuristics (Radford et al., 2019), limit out-of-domain performance (Sharami et al., 2023), and can affect prompting and truncated sampling in unpredictable ways.⁴ Edman et al. (2022) downsamples characters using subword information and has shown promising results in machine translation tasks. The amount of high-frequency information in images and audio means that tokenization cannot be performed losslessly, and instead clustering (Hsu et al., 2021) or discrete auto-encoders (Ramesh et al., 2021) are used to compress the inputs, which lose information and likely limit generative model performance. Our patches are analogous to traditional lossless tokens, and the Local model performs the role of mapping a hidden state to a distribution over possible patches.

10. Conclusion

We introduced MEGABYTE, a scaleable architecture for modeling long sequences. MEGABYTE outperforms existing byte-level models across a range of tasks and modalities, allowing large models of sequences of over 1 million tokens. It also gives competitive language modeling results with subword models, which may allow byte-level models to replace tokenization. However, the scale of experiments here is far below those of state-of-the-art language models (Brown et al., 2020), and future work should explore scaling MEGABYTE to much larger models and datasets.

⁴For example, whether or not a prompt should end in whitespace depends on details of the underlying subwod algorithm used.

MEGABYTE模型：利用多尺度变换器预测百万字节级的序列数据		
全局尺寸	局部尺寸	bpb
3.5亿参数模型（特征维度=1024，层深度=24）	2.9亿参数模型（特征维度=1024，层深度=20）	1.014
7.6亿参数模型（特征维度=1536，层深度=24）	262百万参数模型（特征维度=1024，层深度=18）	1.002
13亿参数模型（特征维度=2048，层深度=24）	2.18亿参数模型（特征维度=1024，层深度=15）	0.991

表10。PG19数据集上局部模型与全局模型规模对性能的影响。提升全局模型的容量能够改善性能，且这些模型在计算资源与数据规模方面均已做到匹配。

局部模型与全局模型的规模比例。我们在PG19数据集上测试了不同的局部模型/全局模型规模比例。通过将字节整合为补丁，MEGABYTE模型在构建全局模型时所需的标记数仅为局部模型时的 P 倍——这让我们能够在不增加成本的前提下提升全局模型的规模。研究表明，当全局模型的参数数量多于局部模型时，给定的计算预算能够得到最优化利用。这一趋势在所有数据类型以及不同的块大小下均保持一致。

9. 相关工作

以往的研究致力于提升Transformer架构在处理长序列时的效率，其主要动力在于降低自注意力机制所带来的二次方级计算成本。

Efficient Encoder Models Several related techniques to ours have been developed for transformer encoder architectures but cannot be straightforwardly applied to decoders. In particular, patchifying operations have previously been used in image *encoder* models such as ViT (Dosovitskiy et al., 2020), and down- and up-sampling operations have been used for text encoders (Clark et al., 2022), but such methods cannot be naively applied to decoder-only models without leaking information to future bytes in the same patch. MEGABYTE generalizes these approaches to an efficient decoder model by using a intra-patch transformer to predict each sequence element’s likelihood, and offseting the inputs to the two models to avoid leaking information. Jaegle et al. (2021) which uses self-attention on a shorter latent sequence, and Didolkar et al. (2022) which uses recurrent model to process chunks with k input steps also resemble patchification, but this technique cannot easily be applied to decoder architectures without leaking information to future timesteps.

Efficient Decoder models Improving the efficiency of decoder models is more challenging because of the need to make one prediction per timestep, and not leak information to future timesteps. The most popular approaches can be categorized as (1) chunking sequences into smaller blocks, and propagating information from previous blocks with either recurrence (Dai et al., 2019; Hutchins et al., 2022) or cross-attention (Hawthorne et al., 2022), (2) linear alternatives to attention, which typically involve forms of token-level recurrence (Katharopoulos et al., 2020; Schlag et al., 2021)

or state space models (Gu et al., 2021; Smith et al., 2022; Ma et al., 2022), or (3) sparse approximations of attention (Kitaev et al., 2020; Beltagy et al., 2020; Child et al., 2019; Wu et al., 2022). However, the performance of dense attention means it is typically still chosen for large scale decoders (Touvron et al., 2023; Chowdhery et al., 2022). MEGABYTE takes the alternative approach of decomposing the complete sequence into two shorter sequences, giving sub-quadratic attention. We also note that feedforward networks are the dominant cost in large decoders, not self-attention. Our approach to compressing sequences allows much larger models than would be possible when using large feedforward networks at every timestep.

Tokenization The most common approach to shortening sequence lengths in Transformer decoders is to pre-process the input with a form of tokenization, in which multiple bytes are mapped to a single discrete token from a fixed vocabulary. For text, this can be done losslessly using methods such as BPE (Sennrich et al., 2015) and SentencePiece (Kudo & Richardson, 2018), but these approaches can require language-specific heuristics (Radford et al., 2019), limit out-of-domain performance (Sharami et al., 2023), and can affect prompting and truncated sampling in unpredictable ways.⁴ Edman et al. (2022) downsamples characters using subword information and has shown promising results in machine translation tasks. The amount of high-frequency information in images and audio means that tokenization cannot be performed losslessly, and instead clustering (Hsu et al., 2021) or discrete auto-encoders (Ramesh et al., 2021) are used to compress the inputs, which lose information and likely limit generative model performance. Our patches are analogous to traditional lossless tokens, and the Local model performs the role of mapping a hidden state to a distribution over possible patches.

10. 结论

我们提出了MEGABYTE，这是一种用于建模长序列的可扩展架构。在多种任务和模式下，MEGABYTE的性能均优于现有的字节级模型，使得处理超过100万个标记的庞大序列成为可能。它在与次词模型的对比中也能展现出具备竞争力的语言建模效果，这或许有望让字节级模型取代传统的标记化方法。不过，目前这些实验的规模仍远远低于最先进的语言模型（Brown等人，2020年），未来的研究应当探索将MEGABYTE应用于规模更大的模型和数据集。

⁴例如，提示词是否应以空格结尾，取决于所使用的底层子模型算法的具体设计。

References

Baines, M., Bhosale, S., Caggiano, V., Goyal, N., Goyal, S., Ott, M., Lefaudeux, B., Liptchinsky, V., Rabbat, M., Sheiffer, S., Sridhar, A., and Xu, M. FairScale: A general purpose modular PyTorch library for high performance and large scale training. <https://github.com/facebookresearch/fairscale>, 2021.

Beltagy, I., Peters, M. E., and Cohan, A. Long-former: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

Clark, J. H., Garrette, D., Turc, I., and Wieting, J. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. Transformer-xl: Attentive language models beyond a fixed-length context, 2019. URL <https://arxiv.org/abs/1901.02860>.

Didolkar, A., Gupta, K., Goyal, A., Gundavarapu, N. B., Lamb, A., Ke, N. R., and Bengio, Y. Temporal latent bottleneck: Synthesis of fast and slow processing mechanisms in sequence learning, 2022.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Edman, L., Toral, A., and van Noord, G. Subword-delimited downsampling for better character-level translation, 2022.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., and Leahy, C. The pile: An 800gb dataset of diverse text for language modeling, 2020.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

Hawthorne, C., Jaegle, A., Cangea, C., Borgeaud, S., Nash, C., Malinowski, M., Dieleman, S., Vinyals, O., Botvinick, M., Simon, I., et al. General-purpose, long-context autoregressive modeling with perceiver ar. In *International Conference on Machine Learning*, pp. 8535–8558. PMLR, 2022.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Hsu, W.-N., Bolte, B., Tsai, Y.-H. H., Lakhotia, K., Salakhutdinov, R., and Mohamed, A. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. Block-recurrent transformers. *arXiv preprint arXiv:2203.07852*, 2022.

Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., and Carreira, J. Perceiver: General perception with iterative attention. In *International conference on machine learning*, pp. 4651–4664. PMLR, 2021.

Kalchbrenner, N., Elsen, E., Simonyan, K., Noury, S., Casagrande, N., Lockhart, E., Stimberg, F., van den Oord, A., Dieleman, S., and Kavukcuoglu, K. Efficient neural audio synthesis. *CoRR*, abs/1802.08435, 2018. URL <http://arxiv.org/abs/1802.08435>.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pp. 5156–5165. PMLR, 2020.

Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *ICLR*, 2015.

参考文献

Baines, M., Bhosale, S., Caggiano, V., Goyal, N., Goyal, S., Ott, M., Lefaudeux, B., Liptchinsky, V., Rabbat, M., Sheiffer, S., Sridhar, A. 与 Xu, M. 撰写的《FairScale: 一个用于高性能大规模训练的通用模块化 PyTorch 库》。<https://github.com/facebookresearch/fairscale>, 2021年。

Beltagy, I., Peters, M. E., 和 Cohan, A. 所著的《Long-former: 长文档Transformer架构》。*arXiv*预印本, 编号为*arXiv:2004.05150*, 2020年出版。

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

Clark, J. H., Garrette, D., Turc, I., and Wieting, J. Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. Transformer-xl: Attentive language models beyond a fixed-length context, 2019. URL <https://arxiv.org/abs/1901.02860>.

Didolkar, A., Gupta, K., Goyal, A., Gundavarapu, N. B., Lamb, A., Ke, N. R., and Bengio, Y. Temporal latent bottleneck: Synthesis of fast and slow processing mechanisms in sequence learning, 2022.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Edman, L., Toral, A., and van Noord, G. Subword-delimited downsampling for better character-level translation, 2022.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S., and Leahy, C. The pile: An 800gb dataset of diverse text for language modeling, 2020.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

Hawthorne, C., Jaegle, A., Cangea, C., Borgeaud, S., Nash, C., Malinowski, M., Dieleman, S., Vinyals, O., Botvinick, M., Simon, I., et al. General-purpose, long-context autoregressive modeling with perceiver ar. In *International Conference on Machine Learning*, pp. 8535–8558. PMLR, 2022.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Hsu, W.-N., Bolte, B., Tsai, Y.-H. H., Lakhotia, K., Salakhutdinov, R., and Mohamed, A. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. Block-recurrent transformers. *arXiv preprint arXiv:2203.07852*, 2022.

Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., and Carreira, J. Perceiver: General perception with iterative attention. In *International conference on machine learning*, pp. 4651–4664. PMLR, 2021.

Kalchbrenner, N., Elsen, E., Simonyan, K., Noury, S., Casagrande, N., Lockhart, E., Stimberg, F., van den Oord, A., Dieleman, S., and Kavukcuoglu, K. Efficient neural audio synthesis. *CoRR*, abs/1802.08435, 2018. URL <http://arxiv.org/abs/1802.08435>.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pp. 5156–5165. PMLR, 2020.

D. P. Kingma与J. Adam Ba共同提出了一种用于随机优化的方法。该文发表于2015年的*ICLR*会议上。

MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers

Kitaev, N., Kaiser, Ł., and Levskaya, A. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Kudo, T. and Richardson, J. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226*, 2018.

Ma, X., Zhou, C., Kong, X., He, J., Gui, L., Neubig, G., May, J., and Zettlemoyer, L. Mega: moving average equipped gated attention. *arXiv preprint arXiv:2209.10655*, 2022.

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

Oord, A. v. d., Kalchbrenner, N., and Kavukcuoglu, K. Pixel Recurrent Neural Networks. *ICML*, 4:2611–2620, 1 2016. doi: 10.48550/arxiv.1601.06759. URL <https://arxiv.org/abs/1601.06759v3>.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019.

Press, O., Smith, N. A., and Lewis, M. Shortformer: Better language modeling using shorter inputs. *arXiv preprint arXiv:2012.15832*, 2020.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019.

Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019a.

Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019b.

Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., and Sutskever, I. Zero-shot text-to-image generation. In *International Conference on Machine Learning*, pp. 8821–8831. PMLR, 2021.

Ren, H., Dai, H., Dai, Z., Yang, M., Leskovec, J., Schuurmans, D., and Dai, B. Combiner: Full attention transformer with sparse computation cost, 2021. URL <https://arxiv.org/abs/2107.05768>.

Roy, A., Saffar, M., Vaswani, A., and Grangier, D. Efficient content-based sparse attention with routing transformers, 2020. URL <https://arxiv.org/abs/2003.05997>.

Salimans, T., Karpathy, A., Chen, X., and Kingma, D. P. Pixelcnn++: Improving the pixelcnn with discretized logistic mixture likelihood and other modifications. *CoRR*, abs/1701.05517, 2017. URL <http://arxiv.org/abs/1701.05517>.

Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9355–9366. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/schlag21a.html>.

Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

Sharami, J., Shterionov, D., and Spronck, P. A systematic analysis of vocabulary and bpe settings for optimal fine-tuning of nmt: A case study of in-domain translation. *arXiv preprint arXiv:2303.00722*, 2023.

Smith, J. T., Warrington, A., and Linderman, S. W. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933*, 2022.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Sun, S., Krishna, K., Mattarella-Micke, A., and Iyyer, M. Do long-range language models actually use long-range context? *arXiv preprint arXiv:2109.09115*, 2021.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Trinh, T. H. and Le, Q. V. A simple method for common-sense reasoning. *arXiv preprint arXiv:1806.02847*, 2018.

van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. W., and Kavukcuoglu, K. Wavenet: A generative model for raw audio. *CoRR*, abs/1609.03499, 2016. URL <http://arxiv.org/abs/1609.03499>.

van den Oord, A., Li, Y., Babuschkin, I., Simonyan, K., Vinyals, O., Kavukcuoglu, K., van den Driessche, G., Lockhart, E., Cobo, L. C., Stimberg, F., Casagrande, N., Grewe, D., Noury, S., Dieleman, S., Elsen, E., Kalchbrenner, N., Zen, H., Graves, A., King, H., Walters, T., Belov, D., and Hassabis, D. Parallel wavenet: Fast high-fidelity speech synthesis. *CoRR*, abs/1711.10433, 2017. URL <http://arxiv.org/abs/1711.10433>.

MEGABYTE模型：利用多尺度变换器预测百万字节级序列

N. Kitaev、Ł. Kaiser和A. Levskaya提出了Reformer：一种高效的Transformer架构。*arXiv*预印本 *arXiv:2001.04451*，发表时间为2020年。

Kudo, T. 和 Richardson, J. 撰写的《Sentencepiece：一种用于神经文本处理的简单且与语言无关的子词分词与反分词算法》。*arXiv*预印本 *arXiv:1808.06226*，2018年。

马X、周C、孔X、何J、桂L、Neubig G、May J以及Zettlemoyer L. Mega：带有移动平均机制的门控注意力机制。*arXiv*预印本 *arXiv:2209.10655*，2022年。

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G. 等人。混合精度训练。*arXiv*预印本 *arXiv:1710.03740*，2017年。

Oord, A. v. d., Kalchbrenner, N., 和 Kavukcuoglu, K. 《像素循环神经网络》。*ICML*，第4卷：2611–2620页，2016年1月。doi: 10.48550/arxiv.1601.06759。网址：<https://arxiv.org/abs/1601.06759v3>。

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. 等人。PyTorch：一种命令式风格的高性能深度学习库。载于*NeurIPS*，2019年。

Press, O., Smith, N. A., and Lewis, M. Shortformer: Better language modeling using shorter inputs. *arXiv preprint arXiv:2012.15832*, 2020.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019.

Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019a.

Rae, J. W., Potapenko, A., Jayakumar, S. M., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019b.

Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., Chen, M., and Sutskever, I. Zero-shot text-to-image generation. In *International Conference on Machine Learning*, pp. 8821–8831. PMLR, 2021.

Ren, H., Dai, H., Dai, Z., Yang, M., Leskovec, J., Schuurmans, D., and Dai, B. Combiner: Full attention transformer with sparse computation cost, 2021. URL <https://arxiv.org/abs/2107.05768>.

Roy, A., Saffar, M., Vaswani, A., and Grangier, D. Efficient content-based sparse attention with routing transformers, 2020. URL <https://arxiv.org/abs/2003.05997>.

Salimans, T., Karpathy, A., Chen, X., 和 D. P. Kingma. Pixelcnn++：通过离散化的逻辑混合似然函数及其他改进来优化Pixelcnn. *CoRR*，abs/1701.05517，2017年。网址<http://arxiv.org/abs/1701.05517>。

Schlag, I., Irie, K., 和 Schmidhuber, J. 线性变换器其实是高效的权重编程工具。载于 Meila, M. 和 Zhang, T. (编)，第38届国际机器学习会议论文集，属于机器学习研究论文集的第139卷，页码9355–9366。PMLR，2021年7月18–24日。网址 <https://proceedings.mlr.press/v139/schlag21a.html>。

Sennrich, R., Haddow, B., 和 Birch, A. 基于子词单元的稀有词神经机器翻译。*arXiv*预印本 *arXiv:1508.07909*，2015年。

Sharami, J., Shterionov, D.以及Spronck, P. 《面向NMT最优微调的词汇表与BPE参数设置系统分析：领域内翻译的案例研究》。*arXiv*预印本 *arXiv:2303.00722*，2023年。

Smith, J. T., Warrington, A., 和 Linderman, S. W.。《用于序列建模的简化状态空间层数》。*arXiv*预印本 *arXiv:2208.04933*，2022年。

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Sun, S., Krishna, K., Mattarella-Micke, A., and Iyyer, M. Do long-range language models actually use long-range context? *arXiv preprint arXiv:2109.09115*, 2021.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Trinh, T. H. and Le, Q. V. A simple method for common-sense reasoning. *arXiv preprint arXiv:1806.02847*, 2018.

van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. W., and Kavukcuoglu, K. Wavenet: A generative model for raw audio. *CoRR*, abs/1609.03499, 2016. URL <http://arxiv.org/abs/1609.03499>.

van den Oord, A., Li, Y., Babuschkin, I., Simonyan, K., Vinyals, O., Kavukcuoglu, K., van den Driessche, G., Lockhart, E., Cobo, L. C., Stimberg, F., Casagrande, N., Grewe, D., Noury, S., Dieleman, S., Elsen, E., Kalchbrenner, N., Zen, H., Graves, A., King, H., Walters, T., Belov, D., and Hassabis, D. Parallel wavenet: Fast high-fidelity speech synthesis. *CoRR*, abs/1711.10433, 2017. URL <http://arxiv.org/abs/1711.10433>.

Wu, Y., Rabe, M. N., Hutchins, D., and Szegedy, C. Memorizing transformers. *arXiv preprint arXiv:2203.08913*, 2022.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, V., Mihaylov, T., Ott, M., Shleifer, S., Shuster, K., Simig, D., Koura, S., Sridhar, A., Wang, T., Zettlemoyer, L., and Ai, M. OPT: Open Pre-trained Transformer Language Models. 5 2022a. doi: 10.48550/arxiv.2205.01068. URL <https://arxiv.org/abs/2205.01068v4>.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022b.

吴阳、M. N. Rabe、Hutchins以及C. Szegedy。《记忆Transformer》。*arXiv*预印本 *arXiv:2203.08913*，2022年。

张S.、Roller S.、Goyal N.、Artetxe M.、陈M.、陈S.、Dewan C.、Diab M.、李X.、林V.、Mihaylov T.、Ott M.、Shleifer S.、Shuster K.、Simig D.、Koura S.、Sridhar A.、王T.、Zettlemoyer L.以及Ai M.。OPT：开源预训练Transformer语言模型。5 2022a。doi: 10.48550/arxiv.2205.01068。网址 <https://arxiv.org/abs/2205.01068v4>。

张S.、Roller S.、Goyal N.、Artetxe M.、陈M.、陈S.、Dewan C.、Diab M.、李X.、林X.V.等。Opt：开放预训练的Transformer语言模型。*arXiv*预印本 *arXiv:2205.01068*，2022b。

A. Appendices

A.1. Training Details

To ensure stable training, we applied gradient clipping with a maximum norm of 1.0 and used the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$ (Kingma & Ba, 2015). We used the built-in polynomial decay learning rate scheduler in MetaSeq with 500 warmup updates and the end learning rate set to 0. All models are trained with pre-norm and using ReLU activation. We apply a dropout of 0.1 throughout, but we do not apply any dropout to embeddings. We also use weight decay of 0.1. To initialize the weights, we use a variant based on Megatron-LM codebase, which involves using a normal distribution with a mean of zero and a standard deviation of 0.006. We truncate this normal distribution within two standard deviations and observed substantial gain in both training stability and performance.

A.2. Model Details

As discussed in Section 4.1, we conduct experiments using a fixed compute and data budget across all models to focus our comparisons solely on the model architecture rather than training resources. To achieve this, we adjust model hyperparameters within each architecture so that the time taken for a single update is matched and then train all models for the same number of updates. We list all of model details in Table 11 and Table 12.

	Model	#L	d_{model}	#H	d_{head}
S1	125M	12	768	12	64
S2	350M	24	1024	16	64
S3	760M	24	1536	16	96
S4	1.3B	24	2048	32	64
S5	2.7B	32	2560	32	80
S6	6.7B	32	4096	32	128

Table 11. **Common Model architecture details by size.** For each model size, we show the number of layers (#L), the embedding size (d_{model}), the number of attention heads (#H), the dimension of each attention head (d_{head}).

A. 附录

A.1. 训练细节

To ensure stable training, we applied gradient clipping with a maximum norm of 1.0 and used the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$ (Kingma & Ba, 2015). We used the built-in polynomial decay learning rate scheduler in MetaSeq with 500 warmup updates and the end learning rate set to 0. All models are trained with pre-norm and using ReLU activation. We apply a dropout of 0.1 throughout, but we do not apply any dropout to embeddings. We also use weight decay of 0.1. To initialize the weights, we use a variant based on Megatron-LM codebase, which involves using a normal distribution with a mean of zero and a standard deviation of 0.006. We truncate this normal distribution within two standard deviations and observed substantial gain in both training stability and performance.

A.2. 模型架构详情

如第4.1节所述，我们在所有模型中均采用固定的计算资源与数据量进行实验，这样就能将比较重点完全集中在模型架构而非训练资源上。为此，我们会调整每种架构下的模型超参数，使得单次参数更新所需的时间保持一致，随后让所有模型执行相同次数的参数更新。所有模型相关的详细信息均列于11表格12中。

模型#L		$d_{\text{模型}}$	#H	d_{head}	
S1	125M	12	768	12	64
S2	350M	24	1024	16	64
S3	760M	24	1536	16	96
S4	1.3B	24	2048	32	64
S5	2.7B	32	2560	32	80
S6	6.7B	32	4096	32	128

表11. 按规模分类的常见模型架构参数。对于每种模型规模，我们会列出其层深度（#L）、嵌入维度(d)、注意力头数量（#H）以及每个注意力头的维度(d)。(d_{model})，注意力头数量（#H），以及每个注意力头的维度（ d_{head} ）。

MEGABYTE: Predicting Million-byte Sequences with Multiscale Transformers					
Model	(Global) Size	Local Size	BS	LR	Context Length (in bytes)
arXiv					
Transformer	320M (D=1024, L=22)	N/A	72	2.00E-04	1,024
Perceiver AR	248M (D=1024, L=17)	N/A	72	2.00E-04	8,192 (1024 latents)
MEGABYTE	758M (D=2048, L=14)	262M (D=1024, L=18)	48	2.00E-04	8,192 (patch size 8)
<i>w/o Local model</i>	2.3B (D=2560, L=20)	N/A	48	1.50E-04	8,192 (patch size 4)
<i>w/o global model</i>	N/A	350M (D=1024, L=24)	192	2.00E-04	8,192 (patch size 8)
<i>w/o cross-patch Local model</i>	921M (D=2048, L=17)	350M (D=1024, L=24)	48	2.00E-04	8,192 (patch size 8)
<i>w/ CNN encoder</i>	704M (D=2048, L=13)	262M (D=1024, L=18)	48	2.00E-04	8,192 (patch size 8)
Image task 64 (Table 3)					
MEGABYTE	2.7B (D=2560, L=32)	350M (D=1024, L=24)	2	2.00E-04	12,288 (patch size 12)
Image task 64 (Table 5)					
Transformer	760M (D=1536, L=24)	N/A	512	3.00E-04	2,048
Perceiver AR	227M (D=1024, L=16)	N/A	512	3.00E-04	12,288 (1024 latents)
MEGABYTE	1.3B (D=2048, L=24)	1.3B (D=2048, L=24)	256	3.00E-04	12,288 (patch size 12)
Image task 256					
Transformer	62M (D=768, L=6)	N/A	1536	2.00E-04	1,024
Perceiver AR	62M (D=768, L=6)	N/A	256	2.00E-04	8,192 (768 latents)
MEGABYTE	125M (D=768, L=12)	125M (D=768, L=12)	16	2.00E-04	196,608 (patch size 192)
<i>w/o local model</i>	2.7B (D=4096, L=32)	N/A	16	2.00E-04	196,608 (patch size 48)
<i>w/o global model</i>	125M (D=768, L=12)	125M (D=768, L=12)	16	2.00E-04	196,608 (patch size 192)
<i>w/o cross-patch Local model</i>	250M	156M (D=768, L=15)	16	2.00E-04	196,608 (patch size 192)
<i>w/ CNN encoder</i>	125M (D=768, L=12)	125M (D=768, L=12)	16	2.00E-04	196,608 (patch size 192)
Image task 640					
Transformer	83M (D=768, L=8)	N/A	4800	3.00E-04	1,024
Perceiver AR	62M (D=768, L=6)	N/A	2048	3.00E-04	4,096 (1024 latents)
MEGABYTE	125M (D=768, L=12)	83M (D=768, L=8)	32	3.00E-04	1,228,800 (192 patch size)
audio					
Transformer	135M (D=768, L=13)	N/A	2048	2.00E-04	1024
Perceiver AR	62M (D=768, L=6)	N/A	384	2.00E-04	8,192 (1024 latents)
MEGABYTE	350M (D=1024, L=24)	125M (D=768, L=12)	256	2.00E-04	524,288 (32 patch size)
<i>w/o local model</i>	2.7B (D=4096, L=32)	125M (D=768, L=12)	256	2.00E-04	524,288 (32 patch size)
<i>w/o global model</i>	350M (D=1024, L=24)	125M (D=768, L=12)	256	2.00E-04	524,288 (32 patch size)
<i>w/o cross-patch Local model</i>	350M (D=1024, L=24)	146M (D=768, L=14)	256	2.00E-04	524,288 (32 patch size)
<i>w/ CNN encoder</i>	350M (D=1024, L=24)	125M (D=768, L=12)	256	2.00E-04	524,288 (32 patch size)

Table 12. **Model architecture details.** We report the model size, the embedding size (D), number of layaers(L), total batch size (BS), learning rate(LR), and context length. When we vary the number of model layers from the standard amount for the given size (Table 11), we note this accordingly. For PerceiverAR models, we note the number of latents used, and for MEGABYTE models we note the patch sizes.

B. Pseudocode

Listing 1. Pseudocode of Megabyte model

```
class MegaByteDecoder:
    def __init__(
        self,
        global_args,
        local_args,
        patch_size,
    ):
        self.pad = 0
        self.patch_size = patch_size
        self.globalmodel = TransformerDecoder(global_args)
        self.localmodel = TransformerDecoder(local_args)
```

MEGABYTE模型：利用多尺度变换器预测百万字节级的序列数据					
模型	（全局）尺寸	局部尺寸	BS	LR	上下文长度（以字节计）
arXiv数据库					
Transformer架构	3.2亿（特征维度=1024, 层深度=22）	N/A	72	0.0002	1,024
Perceiver AR系统	2.48亿（特征维度=1024, 层深度=17）	N/A	72	0.0002	8192（1024个潜在特征）
MEGABYTE模型	758百万参数模型（特征维度为=2048, 层深度为=14）	262百万参数模型（特征维度为=1024, 层深度为=18）	48	0.0002	8192（8像素块大小）
无需局部模型	23亿（特征维度=2560, 层深度=20）	N/A	48	1.50E-04	8192（4像素块大小）
无需全局模型	N/A	3.5亿参数模型（特征维度=1024, 层深度=24）	192	0.0002	8192（8像素块大小）
不含跨块局部模型的结构	921M（特征维度=2048, 层深度=17）	3.5亿参数模型（特征维度=1024, 层深度=24）	48	0.0002	8192（8像素块大小）
配备卷积神经网络编码器	704百万参数模型（特征维度=2048, 层深度=13）	262百万参数模型（特征维度=1024, 层深度=18）	48	0.0002	8192（8像素块大小）
图像任务64（表 3）					
MEGABYTE模型	27亿参数模型（特征维度=2560, 层深度=32）	3.5亿参数模型（特征维度=1024, 层深度=24）	2	0.0002	12288（12像素块大小）
图像任务64（表格 5）					
Transformer架构	760M（特征维度=1536, 层深度=24）	N/A	512	3.00E-04	2048
Perceiver AR系统	227M（特征维度=1024, 层深度=16）	N/A	512	3.00E-04	12288（1024个潜在特征）
MEGABYTE模型	1.3B（特征维度=2048, 层深度=24）	12.288亿（特征维度=2048, 层深度=24）	256	3.00E-04	12288（12像素块大小）
图像任务256					
Transformer架构	6200万参数（特征维度=768, 层深度=6）	N/A	1536	0.0002	1024
Perceiver AR系统	6200万参数（特征维度=768, 层深度=6）	N/A	256	0.0002	8192（768个潜在特征）
MEGABYTE模型	1.25亿参数模型（特征维度=768, 层深度=12）	1.25亿参数模型（特征维度=768, 层深度=12）	16	0.0002	196608（块大小为192）
不使用局部模型	27亿参数模型（特征维度=4096, 层深度=32）	N/A	16	0.0002	196608（块大小为48）
不使用全局模型	1.25亿参数模型（特征维度=768, 层深度=12）	1.25亿参数模型（特征维度=768, 层深度=12）	16	0.0002	196608（块大小为192）
无跨块局部模型	250M	1.56亿（特征维度=768, 层深度=15）	16	0.0002	196608（块大小为192）
配备卷积神经网络编码器	1.25亿参数模型（特征维度=768, 层深度=12）	1.25亿参数模型（特征维度=768, 层深度=12）	16	0.0002	196608（块大小为192）
图像任务640					
Transformer架构	6200万参数（特征维度=768, 层深度=8）	N/A	4800	3.00E-04	1,024
Perceiver AR系统	6200万参数（特征维度=768, 层深度=6）	N/A	2048	3.00E-04	4,096（1024个潜在特征）
MEGABYTE模型	1.25亿参数模型（特征维度=768, 层深度=12）	83M（特征维度=768, 层深度=8）	32	3.00E-04	1,228,800（块大小为192）
音频					
Transformer架构	135M（特征维度=768, 层深度=13）	N/A	2048	2.00E-04	1024
Perceiver AR系统	6200万参数模型（特征维度=768, 层深度=6）	N/A	384	0.0002	8192（1024个潜在特征）
MEGABYTE模型	3.5亿参数模型（特征维度=1024, 层深度=24）	1.25亿参数模型（特征维度=768, 层深度=12）	256	0.0002	524288（块大小为32）
不使用局部模型	27亿参数模型（特征维度=4096, 层深度=32）	1.25亿参数模型（特征维度=768, 层深度=12）	256	0.0002	524288（块大小为32）
不使用全局模型	3.5亿参数模型（特征维度=1024, 层深度=24）	1.25亿参数模型（特征维度=768, 层深度=12）	256	0.0002	524288（块大小为32）
不采用跨块局部模型的情况	3.5亿参数模型（特征维度=1024, 层深度=24）	1.46亿参数模型（特征维度=768, 层深度=14）	256	0.0002	524288（块大小为32）
采用卷积神经网络编码器的结构	3.5亿参数模型（特征维度=1024, 层深度=24）	1.25亿参数模型（特征维度=768, 层深度=12）	256	0.0002	524288（块大小为32）

表12. **模型架构详情。**我们在此报告模型的规模、嵌入维度（D）、层数（L）、总批量大小（BS）、学习率（LR）以及上下文长度。当我们将模型层数调整为该规模对应的标准值之外的数值时（详见表11），会对此进行特别说明。对于PerceiverAR模型，我们会注明所使用的潜在特征数量；而对于MEGABYTE模型，则会列出其对应的块大小。

B. 伪代码

列表1.MEGABYTE模型的伪代码

```
class MegaByteDecoder:
    def __init__(
        self,
        global_args,
        local_args,
        patch_size,
    ):
        self.pad = 0
        self.patch_的大小为 = , 补丁更新_的大小也为该值。
        self.globalmodel = 采用Transformer架构作为解码器，其输入参数为_。
        self.localmodel = 同样基于Transformer架构构建解码器，但使用_作为本地参数。
```

```
def forward(
    self,
    bytes,
):
    bytes_global, bytes_local = self.prepare_input(bytes)

    global_bytes_embedded = self.globalmodel.embed(bytes_global)
    global_in = rearrange(
        global_bytes_embedded,
        "b (t p) e -> b t (p e)",
        p=self.patch_size,
    )
    global_output = self.globalmodel(global_in)

    global_output_reshaped = rearrange(
        global_output,
        "b t (p e) -> (b t) p e",
        p=self.patch_size,
    )
    local_bytes_embedded = self.localmodel.embed(bytes_local)
    local_in = local_bytes_embedded + global_output_reshaped
    local_output = self.localmodel(local_in)

    batch_size = bytes_global.shape[0]
    x = rearrange(local_output, "(b t) l v -> b (t l) v", b=batch_size)
    return x

def prepare_input(self, bytes):
    padding_global = bytes.new(bytes.shape[0], self.patch_size).fill_(self.pad)
    bytes_global = torch.cat((padding_global, bytes[:, : -self.patch_size]), -1)

    bytes_input = rearrange(bytes, "b (t p) -> (b t) p", p=self.patch_size)
    padding_local = bytes_input.new(bytes_input.shape[0], 1).fill_(self.pad)
    bytes_local = torch.cat((padding_local, bytes_input[:, :-1]), -1)

    return bytes_global, bytes_local
```

C. PerceiverAR Implementation

To reproduce PerceiverAR in a compute-controlled setting we extended the standard transformer implementation in metaseq with an additonal cross attention layer to compute the latents and match the architecture of PerceiverAR. We trained the model by sampling random spans from each text, matching the procedure used in the PerceiverAR codebase. To be consistent with the original work, we use sliding window evaluation with a stride of $num_latents/2$ unless otherwise noted. In several cases we used the standard metaseq implementation as opposed to specific techniques reported in the original paper: 1) we used standard attention dropout instead of cross-attention dropout 2) We did not implement chunked attention. We verified our implementation by reproducing the ”Standard Ordering” experiments in Table 5 of the Perceiver AR paper. After carefully matching context size, number of latents, the amount of data and training steps used and learning rate, we achieved 3.53 bpb vs 3.54 reported in the original paper.

D. More results

D.1. Patch scan Implementation

Images have a natural structure, containing a grid of $n \times n$ pixels each composed of 3 bytes (corresponding to color channels). We explore two ways of converting images to sequences for modeling (see Figure 6). Firstly, *raster scan* where the pixels are linearized into 3 bytes and concatenated row-by-row. Secondly, *patch scan* where we create patches of shape $p \times p \times 3$ bytes where $p = \sqrt{\frac{P}{3}}$, and then use a raster scan both within and between patches. Unless otherwise specified, MEGABYTE models use *patch scan* for image data.

```
def forward(
    self,
    字节,
):
    字节_全局字节_局部 = 自身。准备_输入(字节)

    对全局_字节_数据进行嵌入操作 = , 得到self.globalmodel.embed字节_全局的结果
    将全局_in = 中的元素重新排列
        全局范围内对_字节_的数据进行了嵌入操作。
        "b (t p) e -> b t (p e)",
        该补丁更新的参数数量为=, 自适应调
    )
    全局输出结果由 = 个自适应生成的元素组成, 这些元素是通过调用global_in对应的
    重排与整形处理后得到。
    最终的全局输出结果为。
    "b t (p e) -> (b t) p e",
    p=self.batch_大小的值。
    )
    local_字节_经过 = self.localmodel嵌入处理后的结果为_local。
    local_输入 = local_字节_经过 + global_输出_重整形后得到结果。
    local_的输出结果为 = self.localmodel处理_输入后的值。

    batch_大小为 = 字节_, 对应global的形状为[0]。
    x =重新排列(局部_输出), (b t)层深度lv -> b (t l) v", h=批量_大小)
    返回 x

def prepare_input(self, 字节):
    使用_全局 = 字节通过PyTorch拼接操作对[0]形状的数值进行填充。 将_大小的补丁更新后的数组用_填充为自定义填充值。
    对_字节的数组执行 = 全局操作, 即将_中的填充值与原数组合并。 字节_的结果为减去_大小后的值。 -1)

    字节数量_: 输入 = , 需对字节数据进行重新排列。 "b (t p) -> (b t) p", p=self.patch_大小)
    添加填充_于局部 = 区域, 生成字节数量_的新输入数据, 其形状为_[0]。 1).使用_方法将_设置为自定义填充值[self.pad]。
    字节数量_: 局部 = 区域, 通过PyTorch拼接操作{torch.cat}合并填充后的数 字节数量_ 的输入 [:-1])。 -1)

    返回 字节_全局, 字节_局部
```

C. PerceiverAR模型实现

To reproduce PerceiverAR in a compute-controlled setting we extended the standard transformer implementation in metaseq with an additonal cross attention layer to compute the latents and match the architecture of PerceiverAR. We trained the model by sampling random spans from each text, matching the procedure used in the PerceiverAR codebase. To be consistent with the original work, we use sliding window evaluation with a stride of $num_latents/2$ unless otherwise noted. In several cases we used the standard metaseq implementation as opposed to specific techniques reported in the original paper: 1) we used standard attention dropout instead of cross-attention dropout 2) We did not implement chunked attention. We verified our implementation by reproducing the ”Standard Ordering” experiments in Table 5 of the Perceiver AR paper. After carefully matching context size, number of latents, the amount of data and training steps used and learning rate, we achieved 3.53 bpb vs 3.54 reported in the original paper.

D. 更多结果

D.1. 数据块扫描实现

图像本身具有天然的结构, 由若干网格状的 $n \times n$ 像素组成, 每个像素由3字节构成 (对应不同的颜色通道)。我们研究了两种将图像转换为序列以用于建模的方法 (见图6)。第一种方法是光栅扫描, 即把像素按顺序线性化为3字节的数据, 并逐行拼接起来。第二种方法是补丁扫描, 我们会先创建大小为 $p \times p \times 3$ 字节的补丁块, 其中 $p = \sqrt{\frac{P}{3}}$, 随后在每个补丁块内部以及不同补丁块之间都采用光栅扫描的方式进行处理。除非另有说明, MEGABYTE模型在处理图像数据时都会使用补丁扫描方式。

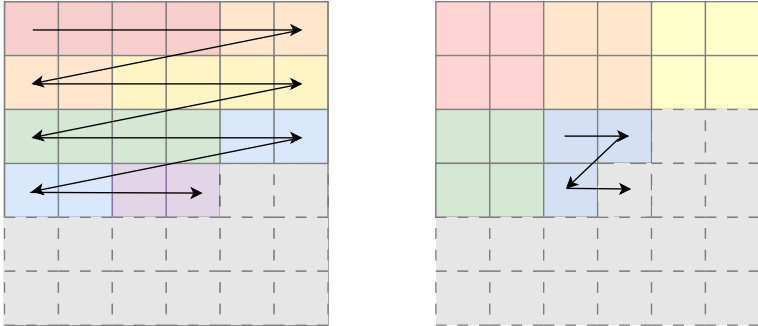


Figure 6. Two ways to model 2D data sequentially. Left, raster scan, by taking bytes row by row and left to right; right, patch scan, where we first split an image into patches, and do raster scan across patches and within a patch. (T=36, K=9, P=4).

D.2. Patch scan vs Raster scan

The patch scan method is inspired by recent works in Vision Transformers (Dosovitskiy et al., 2020), and it is more effective than raster scan for modeling image sequencing. We found it improves both MEGABYTE and Perceiver AR.

	(Global) Size	Local Size	context	bpb
MEGABYTE (patch scan)	62M (D=768, L=6)	N/A	8,192 (768 latents)	3.158
MEGABYTE (raster scan)	62M (D=768, L=6)	N/A	8,192 (768 latents)	3.428
Perceiver AR (patch scan)	125M (D=768, L=12)	125M (D=768, L=12)	196,608 (patch size 192)	3.373
Perceiver AR (raster scan)	125M (D=768, L=12)	125M (D=768, L=12)	196,608 (patch size 192)	3.552

Table 13. ImageNet256 performance with patch scan vs raster scan for MEGABYTE and Perceiver AR.

D.3. Longer sequence modeling

For our pg19 scaling experiment, we also use longer context length for MEGABYTE. The results are shown in Table 14. With longer sequence, we didn’t observer further improvement, consistent with findings in Hawthorne et al. (2022). We think we will benefit more from longer sequence when we futher scale up the model size and data.

	context	bpb
MEGABYTE	8,192 (patch size 8)	0.8751
MEGABYTE	16,384 (patch size 8)	0.8787

Table 14. Longer sequence for PG19 dataset. For both experiments, we set global model as 1.3b, local model as 350m, and MEGABYTE patch size as 8.

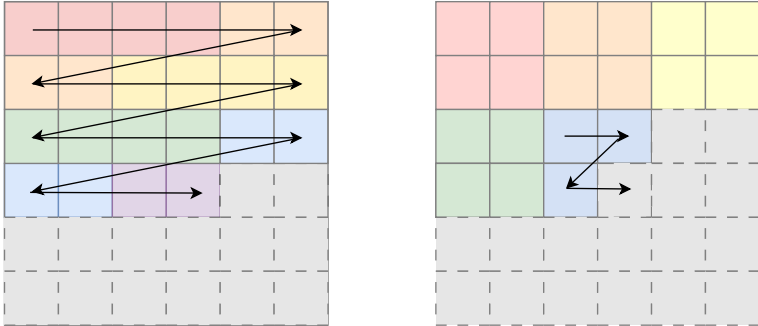


图6。两种建模方式 2D数据依次呈现：左侧为光栅扫描方式，即按从上到下、从左到右的顺序提取字节；右侧为补丁扫描方式，其 e 我们首先将图像分割为多个Pa 对这些特征维度进行操作，并对补丁之间以及单个补丁内部执行光栅扫描。（T=36, K=9, P=4）

D.2. 数据块扫描与光栅扫描的对比

数据块扫描方法的理念源自近期关于视觉变换器的研究（Dosovitskiy等人，2020年），在构建图像序列模型时，它的效果优于光栅扫描。我们发现它能够提升MEGABYTE模型与Perceiver AR系统的性能。

	（全局）尺寸	局部尺寸	上下文	bpb
MEGABYTE模型（数据块扫描）	6200万参数（特征维度=768，层深度=6）	N/A	8192（768个潜在特征）	3.158
MEGABYTE模型（光栅扫描）	6200万参数（特征维度=768，层深度=6）	N/A	8192（768个潜在特征）	3.428
Perceiver AR系统（数据块扫描）	1.25亿参数模型（特征维度=768，层深度=12）	1.25亿参数模型（特征维度=768，层深度=12）	196608（块大小为192）	3.373
Perceiver AR系统（光栅扫描）	1.25亿参数模型（D=768, L=12）	1.25亿参数模型（D=768, L=12）	196608（块大小为192）	3.552

表13。MEGABYTE模型与Perceiver AR系统在采用数据块扫描与光栅扫描两种方式处理ImageNet256数据集时的性能对比。

D.3. 更长序列的建模

在针对PG19数据集的扩展实验中，我们对M也采用了更长的上下文长度EGABYTE。相关结果展示在表14中。随着序列长度的增加，并未观察到进一步的提升，这一现象与Hawthorne等人(2022年)的研究结论一致。我们认为，当进一步增大模型规模和数据量时，更长的序列长度才会带来更大的收益。

	上下文	bpb
MEGABYTE模型	8192（即8像素块大小）	0.8751
MEGABYTE模型	16384（即8像素块大小）	0.8787

表14。针对PG19数据集的更长序列情况。在两项实验中，我们设定的全局模型参数数量为13亿，局部模型参数数量为3.5亿参数模型，同时使用的还有MEGABYTE模型。块大小为8。